

Unidad Didáctica

Fundamentos de Consultas de Bases de Datos, Unidad I



Índice de Contenido

Contenido

Introducción	4
Desarrollo de Contenidos	6
Orden de Procesamiento de Consultas.....	6
Tipos de Datos	7
Comandos SQL para manipulación de registros	9
Comando SELECT FROM.....	9
Comando INSERT	13
Comando UPDATE.....	15
Comando DELETE	17
Cláusulas SQL	18
Cláusula WHERE	18
Cláusula TOP	19
Cláusula ORDER BY.....	20
Cláusula DISTINCT.....	22
Cláusula GROUP BY.....	23
Funciones de Agregado	24
Función MAX y MIN	24
Función CEILING y FLOOR.....	25
Función SUM	26
Función AVG.....	27
Función COUNT	28
Función HAVING	30
Operadores Lógicos	31
Operador AND	31

Operador OR	32
Operador IN	33
Operador LIKE	34
Operador NOT	47
Operador BETWEEN	49
Funciones de Texto	51
LEFT / RIGHT	51
LEN	57
LOWER Y UPPER	60
REPLICATE	62
LTRIM Y RTRIM	66
CONCAT	72
Funciones de Fecha	76
DATEADD	76
DATEDIFF	88
DATEPART	101
ISDATE	111
Otras Funciones	115
CAST	115
CONVERT	115
ISNULL	155
Bibliografía y Webgrafía	161
Glosario	162

Introducción

En el vasto mundo de la administración y gestión de bases de datos, existen aspectos fundamentales que desempeñan un papel crucial en el desarrollo de sistemas eficientes y robustos. En este contexto, se abren las puertas al fascinante campo del procesamiento de consultas, donde se orchestra la sinfonía de datos en armonía con las necesidades del usuario. Desde la selección adecuada de comandos SQL para manipular registros, hasta la optimización de cláusulas SQL que permitan extraer información valiosa, este conjunto de herramientas se convierte en el cimiento sobre el cual se construyen soluciones ágiles y funcionales.

Dentro de este universo de datos, otro elemento esencial es la comprensión y manejo de los diversos tipos de datos. Como un alfabeto diverso, estos tipos establecen las reglas para expresar y representar la información almacenada. Desde números y cadenas de texto, hasta fechas y lógica booleana, cada tipo de dato abre nuevas posibilidades para organizar, analizar y relacionar la información de manera significativa. Abrazar esta variedad de datos es un reto apasionante, pues permite una mejor comprensión del comportamiento de los registros y posibilita la realización de consultas y análisis más precisos y ajustados a las necesidades específicas.

En la constante búsqueda de perfección y eficiencia, las funciones de agregado, lógicas, de texto y de fecha surgen como aliados imprescindibles. Estas funciones, verdaderas herramientas maestras, se convierten en el ingenio detrás del procesamiento de datos a gran escala. Ya sea para resumir información mediante sumatorias y promedios, para tomar decisiones basadas en condiciones lógicas, para manipular y transformar cadenas de texto, o incluso para rastrear y gestionar fechas relevantes, las funciones se convierten en el motor que impulsa el flujo de datos con precisión y efectividad. Así, adentrarse en el vasto conocimiento de estas funciones abre la puerta a un universo de posibilidades en el análisis y explotación de los datos almacenados en sistemas de gestión de bases de datos.

En resumen, este viaje académico a través del orden de procesamiento de consultas, los tipos de datos, los comandos SQL, las cláusulas, funciones de agregado, operadores lógicos, funciones de texto y de fecha, entre otras funciones, promete ser estimulante y enriquecedor. El dominio de estos temas no solo permitirá desenvolverse con destreza en el manejo de bases de datos, sino también desatará la creatividad para extraer el valor oculto en cada conjunto de información. Así, invito a adentrarnos juntos en este apasionante universo del procesamiento de datos, donde la lógica y la precisión se mezclan con el desafío constante de desvelar el significado que se esconde tras la maraña de números y letras, en pos de optimizar nuestros procesos y tomar decisiones fundamentadas.

Orden de Procesamiento de Consultas

En la mayoría de los lenguajes de programación las sentencias se ejecutan from Top to Bottom, es decir, de arriba hacia abajo. Primero se ejecutarán las sentencias del código que se sitúen más arriba, después la siguiente, después la siguiente y así sucesivamente hasta que se determine el final del archivo o nosotros le dictaminemos el final a mano. Sin embargo Sql Server ejecuta el conjunto de declaraciones en un orden lógico, lo que diferencia de la mayoría de los lenguajes de programación convencionales.

SQL Server es único en esto ya que ejecuta las declaraciones en un orden predefinido conocido como Logical Query Processing Phase y que vamos a explicar a continuación

Logical Query Processing Phase

Cuando escribes una serie de comandos dentro de una query sql, estos comandos se dividen mediante una fase de procesamiento lógico. Cada fase genera una serie de tablas virtuales que alimentan a la siguiente fase. Por supuesto, dichas tablas virtuales no son visibles por los administradores. Las fases y su orden de ejecución son las siguientes.

1. FROM
2. ON
3. OUTER
4. WHERE
5. GROUP BY
6. CUBE | ROLLUP
7. HAVING
8. SELECT
9. DISTINCT
- 10 ORDER BY
11. TOP

Tipos de Datos

En SQL Server, una columna, variable y parámetro contiene un valor asociado con un tipo, o también conocido como tipo de datos. Un tipo de datos es un atributo que especifica el tipo de datos que estos objetos pueden almacenar. Puede ser un número entero, una cadena de caracteres, un valor monetario, una fecha y hora, etc.

SQL Server proporciona una lista de tipos de datos que puedes usar al momento de definir por ejemplo una columna o declarar una variable. A continuación, se ilustra el sistema de tipos de datos de SQL Server:



Ten en cuenta que SQL Server eliminará los tipos de datos ntext, text e image en su versión futura. Por lo tanto, debes evitar usar estos tipos de datos y usar los tipos de datos nvarchar(max), varchar(max) y varbinary(max) en su lugar.

Tipos de datos numéricos exactos

Los tipos de datos numéricos exactos almacenan números exactos, como números enteros, decimales o monetarios.

- El bit almacena uno de los tres valores siguientes: 0, 1 y NULL
- Los tipos de datos int, bigint, smallint y tinyint almacenan datos enteros.
- Los tipos de datos decimal y numeric almacenan números que tienen precisión y escala fijas. Ten en cuenta que decimal y numeric son sinónimos.
- Los tipos de datos money y smallmoney almacenan valores de moneda.

Tipos de datos numéricos aproximados

El tipo de datos numérico aproximado almacena datos numéricos de punto flotante. A menudo se utilizan en cálculos científicos.

Tipos de datos de fecha y hora

Los tipos de datos de fecha y hora almacenan fechas y datos de hora. Si desarrollas una nueva aplicación, debes utilizar los tipos de datos time, date, datetime2 y datetimeoffset. Porque estos tipos se alinean con el estándar SQL y son más portátiles. Además, time, datetime2 y datetimeoffset tienen más segundos de precisión y datetimeoffset admite la zona horaria.

Tipos de datos de cadenas de caracteres

Los tipos de datos de cadenas de caracteres permiten almacenar datos de longitud fija (char) o de longitud variable (varchar). El tipo de datos de texto puede almacenar datos no Unicode en la página de códigos del servidor.

Tipos de datos de cadena de caracteres Unicode

Los tipos de datos de cadena de caracteres Unicode almacenan datos de caracteres Unicode de longitud fija (nchar) o de longitud variable (nvarchar).

Tipos de datos de cadenas binarias

Los tipos de datos binarios almacenan datos binarios de longitud fija y variable.

[Consultar](#)

Comandos SQL para manipulación de registros

Comando SELECT FROM

La instrucción SELECT en SQL se usa para recuperar datos de una base de datos relacional

Numero de Columnas	Sintaxis SQL
1	SELECT "column_name" FROM "table_name";
Más de una columna	SELECT "column_name1", "column_name2" FROM "table_name";
Todas las columnas	SELECT * FROM "table_name";

"table_name" es el nombre de la tabla donde se almacenan los datos, y "column_name" es el nombre de la columna que contiene los datos que se recuperarán.

Para seleccionar más de una columna, agregue una coma al nombre de la columna anterior y luego agregue el nombre de la columna. Si está seleccionando tres columnas, la sintaxis será,

```
SELECT "column_name1", "column_name2", "column_name3" FROM "table_name";
```

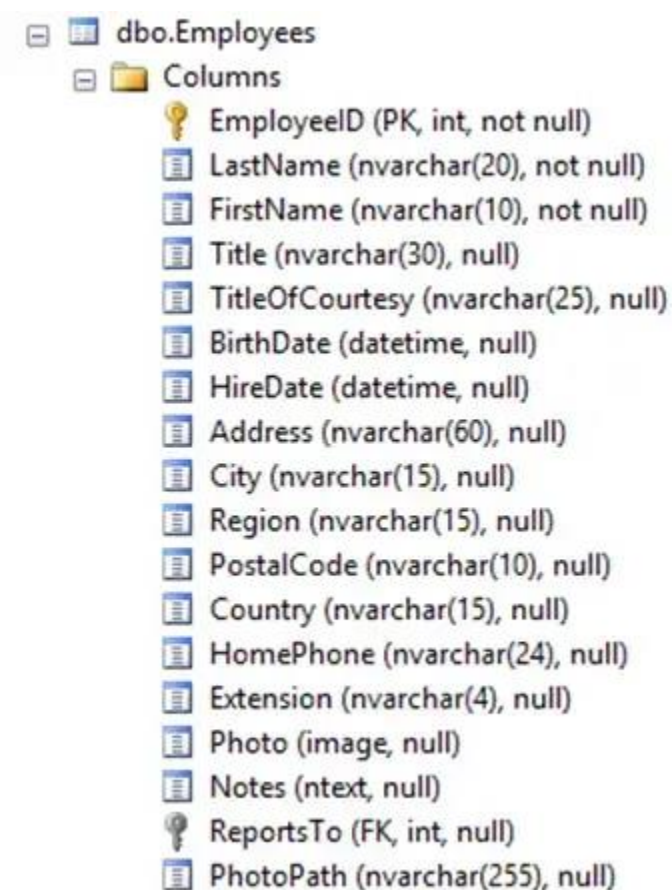
Ten en cuenta que no hay una coma después de la última columna seleccionada.

Ejemplos:

Proporcionaremos ejemplos para cada uno de los siguientes tres casos de uso:

- Recuperar una columna
- Recuperar columnas múltiples
- Recuperar todas las columnas

Usemos la siguiente tabla para ilustrar los tres casos:



Ejemplo 1: Seleccione una columna

Para seleccionar una sola columna, especificamos el nombre de la columna entre SELECT y FROM de la siguiente manera:

```
select FirstName from Employees
```

Resultado:

FirstName
Nancy
Andrew
Janet
Margaret
Steven
Michael
Robert
Laura
Anne

Ejemplo 2: seleccionar múltiples columnas

Podemos usar la instrucción SELECT para recuperar más de una columna. Para seleccionar las columnas FirstName y LastName, usamos el siguiente SQL:

```
select FirstName, LastName from Employees
```

Resultado:

FirstName	LastName
Nancy	Davolio
Andrew	Fuller
Janet	Leverling
Margaret	Peacock
Steven	Buchanan
Michael	Suyama

FirstName	LastName
Robert	King
Laura	Callahan
Anne	Dodsworth

Ejemplo 3: Seleccionar todas las columnas

Hay dos formas de seleccionar todas las columnas de una tabla. El primero es listar todos los nombres de las columnas una por una. La segunda, y la más fácil, es usar el símbolo *. Por ejemplo, para seleccionar todas las columnas de Employees, emitimos el siguiente SQL:

Resultado:

#	FirstName	LastName	Title
1	Nancy	Davolio	Sales Representative
2	Andrew	Fuller	Vice President, Sales
3	Janet	Leverling	Sales Representative
4	Margaret	Peacock	Sales Representative
5	Steven	Buchanan	Sales Manager
6	Michael	Suyama	Sales Representative
7	Robert	King	Sales Representative
8	Laura	Callahan	Inside Sales Coordinator
9	Anne	Dodsworth	Sales Representative

Comando INSERT

El comando INSERT se utiliza para añadir datos a una tabla existente. La tabla en donde los datos se añaden se pasa utilizando un argumento de tipo `nom_sql` o `cadena_sql`. Los argumentos opcionales tipo `ref_columna` permiten definir las columnas en las cuales insertar los valores. Si no se pasa `ref_columna`, los valores se insertarán en el mismo orden que en la base (el primer valor pasado se insertará en la primera columna, el segundo valor en la segunda columna, y así sucesivamente).

Nota: este comando no soporta campos 4D de tipo Objeto.

La palabra clave `VALUES` se utiliza para pasar el o los valor(es) a insertar en la(s) columna(s) especificada(s). Puede pasar una expresión aritmética o `NULL`. Por otra parte, una subconsulta se puede pasar en la palabra clave `VALUES` con el fin de insertar una selección de datos que se pasan como los valores.

El número de valores pasados vía la palabra clave `VALUES` debe coincidir con el número de columnas especificado por el argumento de tipo `ref_columna` pasado y cada uno de ellos también debe coincidir con el tipo de datos de la columna correspondiente o al menos ser convertible a ese tipo de datos.

La palabra clave `INFILE` le permite utilizar el contenido de un archivo externo para especificar los valores de un nuevo registro. Esta palabra clave sólo se debe utilizar con expresiones de tipo `VARCHAR`. Cuando se pasa la palabra clave `INFILE`, el valor expresión aritmética se evalúa como una ruta del archivo, y si se encuentra el archivo, el contenido del archivo se inserta en la columna correspondiente. Sólo los campos de tipo texto o `BLOB` pueden recibir valores de un `INFILE`. El contenido del archivo se transfiere como datos brutos, sin interpretación.

El archivo buscado debe estar en el equipo que aloja el motor SQL, incluso si la consulta viene de un cliente remoto. Del mismo modo, la ruta debe expresarse respetando la sintaxis del sistema operativo del motor de SQL. Puede ser absoluta o relativa.

El comando INSERT es soportado en búsquedas mono y multilíneas. Sin embargo, una instrucción INSERT multilíneas no permite efectuar operaciones UNION y JOIN.

El motor de 4D permite la inserción de valores multilíneas, que pueden simplificar y optimizar el código, en particular, al introducir grandes cantidades de datos. La sintaxis de las inserciones multilíneas es del tipo

```
INSERT INTO {sql_name | sql_string}
[(column_ref, ..., column_ref)]
VALUES(arithmetic_expression, ..., arithmetic_expression), ...,
(arithmetic_expression, ..., arithmetic_expression);
```

Ejemplo 1

Este ejemplo simple permite insertar una selección de la table2 en la table1:

```
INSERT INTO table1 (SELECT * FROM table2)
```

Ejemplo 2

Este ejemplo crea una tabla y luego inserta los valores en ella:

```
CREATE TABLE ACTOR_FANS
(ID INT32, Nom VARCHAR);
INSERT INTO ACTOR_FANS
```

```
(ID, Nom)
```

```
VALUES (1, 'Francis');
```

Ejemplo 3

La sintaxis multilíneas permite evitar la repetición tediosa de líneas:

```
INSERT INTO MiTabla
```

```
(Campo1,Campo2,CampoBol,CampoHora,CampoInfo)
```

```
VALUES
```

```
(1,1,1,'11/01/01','11:01:01','Primera línea'),
```

```
(2,2,0,'12/01/02','12:02:02','Segunda línea'),
```

```
(3,3,1,'13/01/03','13:03:03','Tercera línea'),
```

```
.....
```

```
(7,7,1,'17/01/07','17:07:07','Séptima línea');
```

Nota: no es posible combinar las variables simples y los arrays en la misma instrucción INSERT.

Comando UPDATE

La sentencia UPDATE se utiliza para modificar valores en una tabla.

La sintaxis de SQL UPDATE es:

```
UPDATE nombre_tabla
```

```
SET columna1 = valor1, columna2 = valor2
```

```
WHERE columna3 = valor3
```

La cláusula SET establece los nuevos valores para las columnas indicadas.

La cláusula WHERE sirve para seleccionar las filas que queremos modificar.

Ojo: Si omitimos la cláusula WHERE, por defecto, modificará los valores en todas las filas de la tabla.

Ejemplo del uso de SQL UPDATE

nombre	apellido1	apellido2
ANTONIO	PEREZ	GOMEZ
LUIS	LOPEZ	PEREZ
ANTONIO	GARCIA	BENITO
PEDRO	RUIZ	GONZALEZ

Si queremos cambiar el apellido2 'BENITO' por 'RODRIGUEZ' ejecutaremos:

```
UPDATE personas  
SET apellido2 = 'RODRIGUEZ'  
WHERE nombre = 'ANTONIO'  
AND apellido1 = 'GARCIA'  
AND apellido2 = 'BENITO'
```

Ahora la tabla 'personas' quedará así:

nombre	apellido1	apellido2
ANTONIO	PEREZ	GOMEZ
LUIS	LOPEZ	PEREZ
ANTONIO	GARCIA	RODRIGUEZ
PEDRO	RUIZ	GONZALEZ

Comando DELETE

La sentencia DELETE sirve para borrar filas de una tabla.

La sintaxis de SQL DELETE es:

```
DELETE FROM nombre_tabla
```

```
WHERE nombre_columna = valor
```

Si queremos borrar todos los registros o filas de una tabla, se utiliza la sentencia:

```
DELETE * FROM nombre_tabla;
```

Ejemplo de SQL DELETE para borrar una fila de la tabla personas

nombre	apellido1	apellido2
ANTONIO	PEREZ	GOMEZ
LUIS	LOPEZ	PEREZ
ANTONIO	GARCIA	RODRIGUEZ
PEDRO	RUIZ	GONZALEZ

Si queremos borrar a la persona LUIS LOPEZ PEREZ, podemos ejecutar el comando:

```
DELETE FROM personas  
WHERE nombre = 'LUIS'  
AND apellido1 = 'LOPEZ'  
AND apellido2 = 'PEREZ'
```

La tabla 'personas' resultante será:

nombre	apellido1	apellido2
ANTONIO	PEREZ	GOMEZ
ANTONIO	GARCIA	RODRIGUEZ
PEDRO	RUIZ	GONZALEZ

Cláusulas SQL

Cláusula WHERE

La cláusula WHERE se utiliza para hacer filtros en las consultas, es decir, seleccionar solamente algunas filas de la tabla que cumplan una determinada condición.

El valor de la condición debe ir entre comillas simples ".

Por ejemplo:

Seleccionar las personas cuyo nombre sea ANTONIO

```
SELECT * FROM personas  
WHERE nombre = 'ANTONIO'
```

nombre	apellido1	apellido2
ANTONIO	PEREZ	GOMEZ
ANTONIO	GARCIA	BENITO

Cláusula TOP

La sentencia SQL TOP se utiliza para especificar el número de filas a mostrar en el resultado.

Esta cláusula SQL TOP es útil en tablas con muchos registros, para limitar el número de filas a mostrar en la consulta, y así sea más rápida la consulta, consumiendo también menos recursos en el sistema.

Esta cláusula se especifica de forma diferente según el sistema de bases de datos utilizado.

Cláusula SQL TOP para SQL SERVER

SELECT TOP número

PERCENT nombre_columna

FROM nombre_tabla

Ejemplo SQL TOP

Dada la siguiente tabla 'personas', quiero obtener los 2 primeros valores.

nombre	apellido1	apellido2
ANTONIO	PEREZ	GOMEZ
ANTONIO	GARCIA	RODRIGUEZ
PEDRO	RUIZ	GONZALEZ

SELECT TOP 2 * FROM personas

Obtendríamos el siguiente resultado

nombre	apellido1	apellido2
ANTONIO	PEREZ	GOMEZ
ANTONIO	GARCIA	RODRIGUEZ

Cláusula ORDER BY

ORDER BY se utiliza para ordenar los resultados de una consulta, según el valor de la columna especificada.

Por defecto, se ordena de forma ascendente (ASC) según los valores de la columna.

Si se quiere ordenar por orden descendente se utiliza la palabra DES

```
SELECT nombre_columna(s)
FROM nombre_tabla
ORDER BY nombre_columna(s) ASC|DESC
```

Por ejemplo, en la tabla personas :

nombre	apellido1	apellido2
ANTONIO	PEREZ	GOMEZ
LUIS	LOPEZ	PEREZ
ANTONIO	GARCIA	BENITO

```
SELECT nombre, apellido1
FROM personas
ORDER BY apellido1 ASC
```

Esta es la consulta resultante:

nombre	apellido1
LUIS	LOPEZ
ANTONIO	GARCIA
ANTONIO	PEREZ

Ejemplo de ordenación descendiente (DES)

```
SELECT nombre, apellido1
FROM personas
ORDER BY apellido1 DESC
```

Esta es la consulta resultante:

nombre	apellido1
ANTONIO	PEREZ
ANTONIO	GARCIA
LUIS	LOPEZ

Cláusula DISTINCT

Al realizar una consulta puede ocurrir que existan valores repetidos para algunas columnas. Por ejemplo

```
SELECT nombre FROM personas
```

nombre
ANTONIO
LUIS
ANTONIO

Esto no es un problema, pero a veces queremos que no se repitan, por ejemplo, si queremos saber los nombres diferentes que hay en la tabla personas", entonces utilizaremos DISTINCT.

```
SELECT DISTINCT nombre FROM personas
```

nombre
ANTONIO
LUIS

Cláusula GROUP BY

La cláusula GROUP BY de Access combina registros con valores idénticos en la lista de campos especificados en un único registro. Si en la instrucción SELECT se incluye una función de agregado de SQL, como Sum o Count, se creará un valor de resumen para cada registro.

Sintaxis

SELECT listadecampos

FROM tabla

WHERE criterios

[GROUP BY listadecamposdegrupo]

Una instrucción SELECT que contiene una cláusula GROUP BY consta de las siguientes partes:

Parte	Descripción
<i>listadecampos</i>	Nombre de los campos que se recuperarán con cualquier alias de nombre de campo, funciones de agregado de SQL, predicados de selección (ALL, DISTINCT, DISTINCTROW o TOP) u otras opciones de la instrucción SELECT.
<i>tabla</i>	Nombre de la tabla de la cual se recuperan los registros.
<i>criterios</i>	Criterios de selección. Si la instrucción incluye una cláusula WHERE, el motor de base de datos Microsoft Access agrupa los valores después de aplicar las condiciones WHERE a los registros.

Parte	Descripción
<i>listadecamposdegrupo</i>	Nombres de hasta un máximo de 10 campos utilizados para agrupar registros. El orden de los nombres de campo de <i>listadecamposdegrupo</i> determina los niveles de agrupación desde el nivel más alto al nivel más bajo.

Funciones de Agregado

Función MAX y MIN

Devuelva el mínimo o el máximo de un conjunto de valores contenidos en un campo especificado en una consulta.

Sintaxis

Min(expr)

Max(expr)

El marcador de posición de *expr* representa una expresión de cadena el campo que contiene los datos que desea evaluar o una expresión que realiza un cálculo con los datos de ese campo. Los operandos en *expr* pueden incluir el nombre de un campo de tabla, una constante o una función (que puede ser intrínseca o definida por el usuario, pero no una de las otras SQL de agregado).

Observaciones

Puede usar Min y Max para determinar los valores más pequeños y mayores de un campo en función de la agregación o agrupación especificadas. Por ejemplo, puede usar estas funciones para devolver el costo de transporte más bajo y más alto. Si no se especifica ninguna agregación, se usará toda la tabla.

Puede usar Min y Max en una expresión de consulta y en la propiedad SQL de un objetoQueryDef o al crear un objeto Recordset basado en una consulta SQL consulta.

Ejemplos de consulta

Expresión	Resultados
SELECT Min(Preciounidad) como Expr1 de ProductSales;	Devuelve el precio unitario mínimo del campo "Preciounidad" y se muestra en la columna Expr1.
SELECT Max(Preciounidad) como Expr1 de ProductSales;	Devuelve el precio unitario máximo del campo "Preciounidad" y se muestra en la columna xpr1.
SELECT Max(PrecioUnidad) como MaxPrice de ProductSales;	Devuelve el precio unitario máximo del campo "PrecioUnidad" y se muestra en la columna "Precio Máximo".

Función CEILING y FLOOR

Ambas funciones obtienen un número entero

FLOOR: Devuelve siempre la parte entera del número, no tiene en cuenta los decimales, los obvia.

```
SELECT FLOOR(5.92)
```

Resultado:5

CEILING: Devuelve el siguiente número entero mayor al número indicado.

```
SELECT CEILING(5.12)
```

Resultado:6

```
SELECT CEILING(-5.12)
```

Resultado:-5

Función SUM

Devuelve la suma de un conjunto de valores contenidos en un campo especificado en una consulta.

Sintaxis

```
Suma( expr )
```

El marcador de posición de expr representa una expresión de cadena el campo que contiene los datos numéricos que desea agregar o una expresión que realiza un cálculo con los datos de ese campo. Los operandos en expr pueden incluir el nombre de un campo de tabla, una constante o una función (que puede ser intrínseca o definida por el usuario, pero no una de las otras SQL de agregado).

Observaciones

La función Suma suma los valores de un campo. Por ejemplo, puede usar la función Suma para determinar el costo total de los gastos de transporte.

La función Suma omite los registros que contienen campos Null. En el ejemplo siguiente se muestra cómo puede calcular la suma de los productos de los campos PrecioUnidad y Cantidad:

```
SELECT Sum(UnitPrice * Quantity) AS [Total Revenue]
```

FROM [Order Details];

Puede usar la función Suma en una expresión de consulta. También puede usar esta expresión en la propiedad SQL de un objetoQueryDef o al crear un recordset basado en una SQL consulta.

Función AVG

Calcula la media aritmética de un conjunto de valores contenidos en un campo especificado en una consulta.

Sintaxis

`Avg ( expr )`

El marcador de posición de expr representa una expresión de cadena el campo que contiene los datos numéricos que desea promediar o una expresión que realiza un cálculo con los datos de ese campo. Los operandos en expr pueden incluir el nombre de un campo de tabla, una constante o una función (que puede ser intrínseca o definida por el usuario, pero no una de las otras SQL de agregado).

Observaciones

El promedio calculado por Avg es la media aritmética (la suma de los valores divididos por el número de valores). Puede usar Avg, por ejemplo, para calcular el coste medio del transporte.

La función Avg no incluye ningún campo Null en el cálculo.

Puede usar Avg en una expresión de consulta y en la propiedad SQL de un objeto QueryDef o al crear un objeto conjunto de registros basado en una consulta SQL consulta.

Ejemplos

Expresión	Resultados
SELECT Avg([PrecioUnidad]) AS Expr1 FROM ProductSales;	Devuelve el promedio de todos los valores del campo "PrecioUnidad" de la tabla "Ventas de productos" y se muestra en la columna Expr1.
SELECT Avg([SalePrice]) AS AvgSalePrice, Avg([Discount]) AS AvgDiscount FROM ProductSales;	Devuelve el promedio de los campos "PrecioVenta" y "Descuento" de la tabla ProductSales. Los resultados se muestran en las columnas "AvgSalePrice" y "AvgDiscount", respectivamente, devuelve el promedio de todos los "PrecioVenta" donde la "Cantidad" vendida es mayor de 10. Los resultados se muestran en la columna "AvgSalePrice".
SELECT Abs(Avg([Discount])) AS AbsAverageDiscount FROM ProductSales;	Devuelve el valor absoluto del valor promedio del campo "Descuento" y se muestra en la columna "AbsAverageDiscount".

Función COUNT

La función COUNT devuelve el número de filas de la consulta, es decir, el número de registros que cumplen una determinada condición.

Los valores nulos no serán contabilizados.

Sintaxis de SQL COUNT:

```
SELECT COUNT(columna) FROM tabla
```

Para obtener el número de filas de una tabla

```
SELECT COUNT(*) FROM tabla
```

Para obtener el número de valores distintos de la columna especificada.

```
SELECT COUNT(DISTINCT columna) FROM tabla.
```

Ejemplos de SQL COUNT:

Dada la siguiente tabla 'pedidos'

id	pedido	cliente	precio
1	p1	RUIZ	100
2	p2	PEREZ	300
3	p3	GOMEZ	250
4	p4	RODRIGUEZ	490
5	p5	LOPEZ	60

```
SELECT COUNT(*) FROM pedidos
```

Devolverá el número de filas de la tabla, es decir, 5

```
SELECT COUNT(*) FROM pedidos
```

```
WHERE cliente = 'RUIZ'
```

Devolverá el número de filas del resultado de la consulta, es decir, 1

```
SELECT COUNT(*) FROM pedidos
```

```
WHERE precio > 270
```

Devolverá el número de filas del resultado de la consulta, es decir, 2

Función HAVING

La cláusula HAVING en Access especifica qué registros agrupados se muestran en una instrucción SELECT con una cláusula GROUP BY. Después de que GROUP BY combine los registros, HAVING muestra cualquier registro agrupado por la cláusula GROUP BY que satisfaga las condiciones de la cláusula HAVING.

Sintaxis

```
SELECT listadecampos
```

```
FROM tabla
```

```
WHERE criteriosdeselección
```

```
GROUP BY listadecamposdegrupo
```

```
[HAVING criteriosdegrupo]
```

Una instrucción SELECT que contiene una cláusula HAVING consta de las siguientes partes:

Parte	Descripción
<i>listadecampos</i>	Nombre del campo o campos que se van a recuperar junto con cualquier alias de nombre de campo, funciones de agregado de SQL, predicados

Parte	Descripción
	de selección (ALL, DISTINCT, DISTINCTROW o TOP) u otras opciones de la instrucción SELECT.
<i>tabla</i>	Nombre de la tabla de la cual se recuperan los registros.
<i>criteriosdeselección</i>	Criterios de selección. Si la instrucción incluye una cláusula WHERE, el motor de base de datos Microsoft Access agrupa los valores después de aplicar las condiciones WHERE a los registros.
<i>listadecamposdegrupo</i>	Nombres de hasta un máximo de 10 campos utilizados para agrupar registros. El orden de los nombres de campo de <i>listadecamposdegrupo</i> determina los niveles de agrupación desde el nivel más alto al nivel más bajo.
<i>criteriosdegrupo</i>	Expresión que determina los registros agrupados que se muestran.

Operadores Lógicos

Operador AND

Los operadores AND y OR se utilizan para filtrar resultados con 2 condiciones. El operador AND mostrará los resultados cuando se cumplan las 2 condiciones.

Condición1 AND condición2

El operador OR mostrará los resultados cuando se cumpla alguna de las 2 condiciones.

Condicion1 OR condicion2

En la tabla personas

nombre	apellido1	apellido2
ANTONIO	PEREZ	GOMEZ
ANTONIO	GARCIA	BENITO
LUIS	LOPEZ	PEREZ

La siguiente sentencia (ejemplo AND) dará el siguiente resultado:

```
SELECT * FROM personas  
WHERE nombre = 'ANTONIO'  
AND apellido1 = 'GARCIA'
```

nombre	apellido1	apellido2
ANTONIO	GARCIA	BENITO

Operador OR

La siguiente sentencia (ejemplo OR) dará el siguiente resultado:

```
SELECT * FROM personas  
WHERE nombre = 'ANTONIO'
```


OR apellido1 = 'GARCIA'

nombre	apellido1	apellido2
ANTONIO	PEREZ	GOMEZ
ANTONIO	GARCIA	BENITO

También se pueden combinar AND y OR, como el siguiente ejemplo:

SELECT * FROM personas

WHERE nombre = 'ANTONIO'

AND (apellido1 = 'GARCIA' OR apellido1 = 'LOPEZ')

nombre	apellido1	apellido2
ANTONIO	GARCIA	BENITO

Operador IN

El operador IN permite seleccionar múltiples valores en una cláusula WHERE

Sintaxis SQL IN

SELECT columna

FROM tabla

WHERE columna

IN (valor1, valor2, valor3, .)

Ejemplo SQL IN

Dada la siguiente tabla 'personas'

nombre	apellido1	apellido2
ANTONIO	PEREZ	GOMEZ
ANTONIO	GARCIA	RODRIGUEZ
PEDRO	RUIZ	GONZALEZ

Queremos seleccionar a las personas cuyo apellido1 sea 'PEREZ' o 'RUIZ'

```
SELECT * FROM personas
```

```
WHERE apellido1
```

```
IN ('PEREZ','RUIZ')
```

nombre	apellido1	apellido2
ANTONIO	PEREZ	GOMEZ
PEDRO	RUIZ	GONZALEZ

Operador LIKE

El SQL Like es un tipo de operador lógico que se usa para poder determinar si una cadena de caracteres específica coincide con un patrón específico. Se utiliza normalmente en una sentencia Where para buscar un patrón específico de una columna.

Este operador puede ser de utilidad en los casos donde necesitamos realizar un apareamiento de patrones en vez de iguales o no iguales. El SQL Like se utiliza cuando se desea devolver la fila, si una cadena de caracteres específica coincide con

un patrón específico. El patrón puede ser una combinación de caracteres regulares al igual que caracteres de comodín (*,% , ¿, etc.).

Para poder devolver una fila, los caracteres normales se deben hacer coincidir exactamente con los caracteres especificados en la cadena de caracteres. Los caracteres comodín pueden combinarse con partes arbitrarias de la cadena de caracteres.

Utilicemos la base de datos del ejemplo AdventureWorks y veamos algunos de los operadores SQL Like diferentes “%” y “_”. Comodines

Uso del carácter comodín % (representa cero, uno o varios caracteres)

La siguiente consulta retorna todos los números de teléfono cuyo código de área “415” en la tabla “PersonPhone”:

```
SELECT p.FirstName, p.LastName, ph.PhoneNumber
FROM Person.PersonPhone AS ph
INNER JOIN Person.Person AS p
ON ph.BusinessEntityID = p.BusinessEntityID
WHERE ph.PhoneNumber LIKE '415%'
ORDER by p.LastName;
GO
```

Nótese que el símbolo “415%” se especifica en la cláusula “Where”. Esto significa que SQL Server buscará el número 415 seguido de cualquier cadena de cero o más caracteres. Aquí está el conjunto de resultados:

SQLQuery1.sql - D:\TOP-9JU7UEF\boksi)*

```

1  -- Uses AdventureWorks
2
3  SELECT p.FirstName, p.LastName, ph.PhoneNumber
4  FROM Person.PersonPhone AS ph
5  INNER JOIN Person.Person AS p
6  ON ph.BusinessEntityID = p.BusinessEntityID
7  WHERE ph.PhoneNumber LIKE '415%'
8  ORDER BY p.LastName;
9  GO

```

100 %

Results Messages

	FirstName	LastName	PhoneNumber
1	Ruben	Alonso	415-555-0124
2	Shelby	Cook	415-555-0121
3	Karen	Hu	415-555-0114
4	David	Long	415-555-0123
5	John	Long	415-555-0147
6	Gilbert	Ma	415-555-0138
7	Meredith	Moreno	415-555-0131
8	Alexandra	Nelson	415-555-0174
9	Taylor	Patterson	415-555-0170
10	Gabrielle	Russell	415-555-0197
11	Dalton	Simmons	415-555-0115

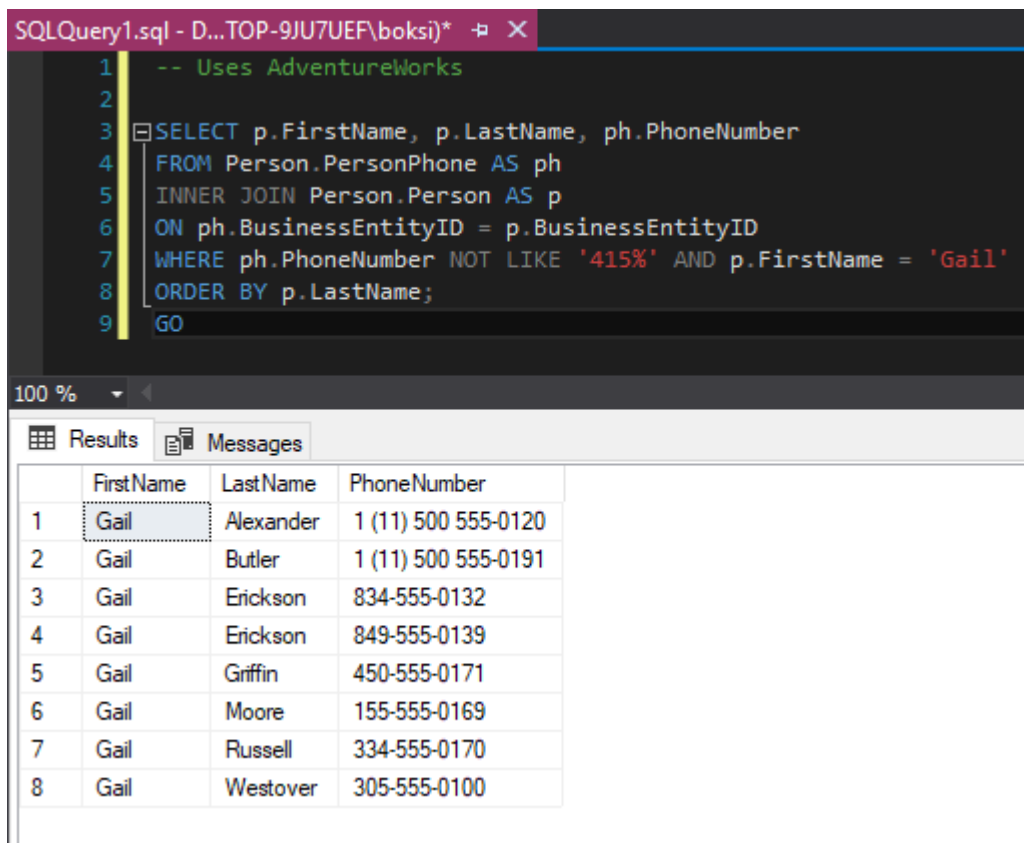
El operador no lógico invierte el valor de cualquier expresión booleana. Por lo tanto, si simplemente especificamos No como con el carácter comodín “%” en la sentencia SQL Like, agrega una condición adicional y colóquela en la misma declaración anterior, de tal forma que deberíamos obtener una consulta como esta:

```

SELECT p.FirstName, p.LastName, ph.PhoneNumber
FROM Person.PersonPhone AS ph
INNER JOIN Person.Person AS p
ON ph.BusinessEntityID = p.BusinessEntityID
WHERE ph.PhoneNumber NOT LIKE '415%' AND p.FirstName = 'Gail'
ORDER BY p.LastName;
GO

```

En esta ocasión, la consulta devolvió todos los registros de la tabla "PersonPhone" que tienen códigos de área distintos de 415:



The screenshot shows a SQL query window titled "SQLQuery1.sql - D...TOP-9JU7UEF\boksi)*". The query is as follows:

```
1  -- Uses AdventureWorks
2
3  SELECT p.FirstName, p.LastName, ph.PhoneNumber
4  FROM Person.PersonPhone AS ph
5  INNER JOIN Person.Person AS p
6  ON ph.BusinessEntityID = p.BusinessEntityID
7  WHERE ph.PhoneNumber NOT LIKE '415%' AND p.FirstName = 'Gail'
8  ORDER BY p.LastName;
9  GO
```

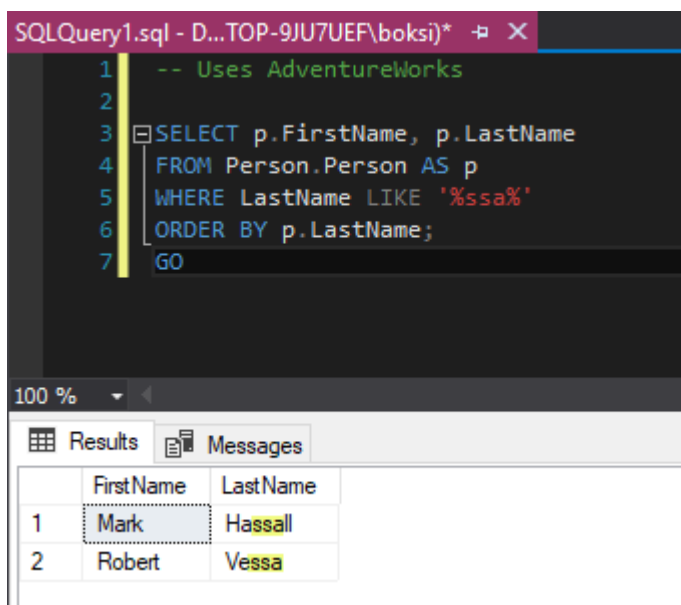
Below the query window, the "Results" tab is active, displaying the following data:

	FirstName	LastName	PhoneNumber
1	Gail	Alexander	1 (11) 500 555-0120
2	Gail	Butler	1 (11) 500 555-0191
3	Gail	Erickson	834-555-0132
4	Gail	Erickson	849-555-0139
5	Gail	Griffin	450-555-0171
6	Gail	Moore	155-555-0169
7	Gail	Russell	334-555-0170
8	Gail	Westover	305-555-0100

Además, supongamos que queremos encontrar todos los registros en los cuales un nombre contiene "ssa" en ellos. Se puede usar la siguiente consulta:

```
SELECT p.FirstName, p.LastName
FROM Person.Person AS p
WHERE LastName LIKE '%ssa%'
ORDER BY p.LastName;
GO
```

Tenga en cuenta que al usar '%' antes y después de "ssa", le estamos indicando a SQL Server que busque todos los registros en los que "Person.Person" tenga caracteres "ssa" y no importa qué otros caracteres estén antes y después de "ssa":



The screenshot shows a SQL query window titled "SQLQuery1.sql - D:\TOP-9JU7UEF\boksi)*". The query is as follows:

```
1  -- Uses AdventureWorks
2
3  SELECT p.FirstName, p.LastName
4  FROM Person.Person AS p
5  WHERE LastName LIKE '%ssa%'
6  ORDER BY p.LastName;
7  GO
```

Below the query window, the "Results" tab is active, displaying the following data:

	FirstName	LastName
1	Mark	Hassall
2	Robert	Vessa

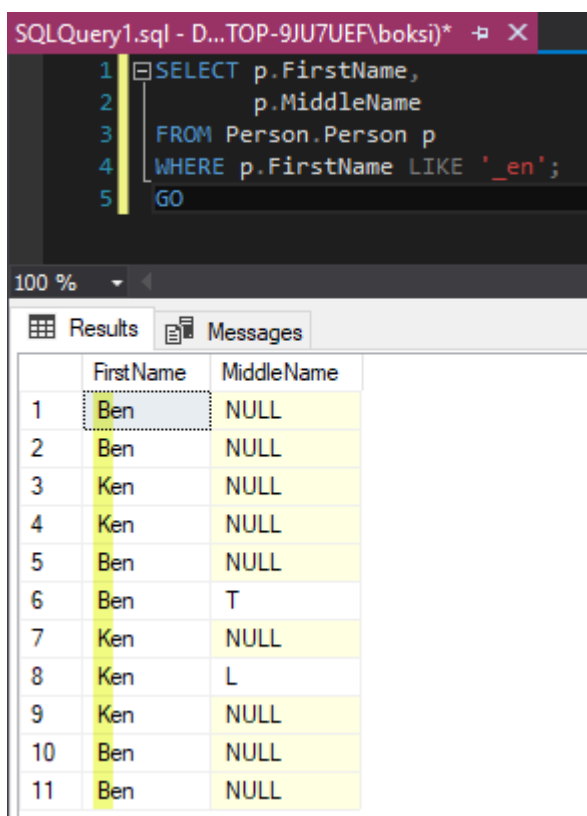
Usando el carácter comodín "_" (representa un solo carácter)

El carácter de subrayado de SQL Like, por ejemplo, se utiliza cuando queremos verificar un solo carácter que puede ser cualquier cosa y proveer el resto de los personajes para nuestro lado.

Supongamos que si deseamos devolver todos los registros en los que el primer carácter de la tabla "Nombre" puede ser cualquier cosa, pero el resto de ellos debe ser "en". Utilice la siguiente consulta:

```
SELECT p.FirstName,  
       p.MiddleName  
FROM Person.Person p  
WHERE p.FirstName LIKE '_en';  
GO
```

Aquí está el conjunto de resultados:



The screenshot shows a SQL query window titled 'SQLQuery1.sql - D:\TOP-9JU7UEF\boksi)*'. The query is as follows:

```
1 SELECT p.FirstName,  
2       p.MiddleName  
3 FROM Person.Person p  
4 WHERE p.FirstName LIKE '_en';  
5 GO
```

Below the query window, the 'Results' tab is active, displaying a table with 11 rows and 2 columns: 'FirstName' and 'MiddleName'. The first row is highlighted with a dotted border.

	FirstName	MiddleName
1	Ben	NULL
2	Ben	NULL
3	Ken	NULL
4	Ken	NULL
5	Ben	NULL
6	Ben	T
7	Ken	NULL
8	Ken	L
9	Ken	NULL
10	Ben	NULL
11	Ben	NULL

Tome en cuenta que también se debe usar una combinación de caracteres comodín al final del patrón de búsqueda. Por ejemplo, para obtener todos los números de teléfono que tienen un código de área que comienza con el número 6 y termina en 2 en la tabla "PersonPhone", use la siguiente consulta:

```
SELECT pp.PhoneNumber  
FROM Person.PersonPhone pp  
WHERE pp.PhoneNumber LIKE '6_2%'  
GO
```

Tome en cuenta que el carácter comodín "%" se utiliza después del carácter de subrayado, ya que el código de área es la primera parte del número de teléfono y existen caracteres adicionales después del valor de la columna

SQLQuery1.sql - D:\TOP-9JU7UEF\boksi)*

```

1 SELECT pp.PhoneNumber
2 FROM Person.PersonPhone pp
3 WHERE pp.PhoneNumber LIKE '6_2%'
4 GO

```

100 %

Results Messages

	PhoneNumber
1	602-555-0100
2	602-555-0112
3	602-555-0120
4	602-555-0122
5	602-555-0130
6	602-555-0134
7	602-555-0165
8	602-555-0169
9	602-555-0169
10	602-555-0171
11	602-555-0171

Usando los corchetes [] (cualquier carácter individual dentro del rango especificado [a-t] o en conjunto [abc])

El operador SQL Like con corchetes es utilizado cuando se quiere tener rango. Supongamos que si queremos encontrar todas las filas donde el primer carácter de "Nombre" comienza con [a-f]. Utilice la siguiente consulta:

```

SELECT BusinessEntityID, FirstName, LastName
FROM Person.Person
WHERE FirstName LIKE '[a-f]%'
GO

```

Como puede verse, hemos utilizado el rango [a-f]%. Eso significa que se debe devolver el primer carácter de "a" a la "f" y después de eso, todos los caracteres están bien porque usamos "%" después:

SQLQuery1.sql - D...TOP-9JU7UEF\boksi)*

```

1 SELECT BusinessEntityID, FirstName, LastName
2 FROM Person.Person
3 WHERE FirstName LIKE '[a-c]%'
4 GO

```

100 %

Results Messages

	BusinessEntityID	FirstName	LastName
1	293	Catherine	Abel
2	16867	Aaron	Adams
3	16901	Adam	Adams
4	16724	Alex	Adams
5	10263	Alexandra	Adams
6	10312	Allison	Adams
7	10274	Amanda	Adams
8	10292	Amber	Adams
9	10314	Andrea	Adams
10	16699	Angel	Adams
11	10299	Bailey	Adams
12	1770	Ben	Adams
13	4194	Blake	Adams
14	305	Carla	Adams
15	16691	Carlos	Adams

Para obtener cualquier carácter individual dentro de un conjunto, utilice el siguiente ejemplo para encontrar a los empleados en la tabla de "Persona" con el nombre de Cheryl o Sheryl:

```

SELECT BusinessEntityID, FirstName, LastName
FROM Person.Person
WHERE FirstName LIKE '[CS]heryl';
GO

```

Esta consulta solo retornara a "Cheryl" en este caso, pero habría devuelto a "Sheryl" también si se tuviera algún registro en la base de datos:

SQLQuery1.sql - D...TOP-9JU7UEF\boksi)*

```

1 SELECT BusinessEntityID, FirstName, LastName
2 FROM Person.Person
3 WHERE FirstName LIKE '[CS]heryl';
4 GO

```

100 %

Results Messages

	BusinessEntityID	FirstName	LastName
1	6184	Cheryl	Alan
2	6177	Cheryl	Alvarez
3	6202	Cheryl	Blanco
4	6208	Cheryl	Carlson
5	6174	Cheryl	Diaz
6	6197	Cheryl	Dominguez
7	6199	Cheryl	Gill
8	6190	Cheryl	Gutierrez
9	6175	Cheryl	Hernandez
10	1065	Cheryl	Herring

A continuación mostramos otro ejemplo cuando actualmente tenemos resultados mixtos:

```

SELECT LastName, FirstName
FROM Person.Person
WHERE LastName LIKE 'Zh[ae]ng'
ORDER BY LastName ASC, FirstName ASC;
GO

```

La consulta previa, muestra los registros de los empleados en la tabla "Persona" con los apellidos de Zheng o Zhang:

SQLQuery1.sql - D:\TOP-9JU7UEF\boksi)*

```

1 SELECT LastName, FirstName
2 FROM Person.Person
3 WHERE LastName LIKE 'Zh[ae]ng'
4 ORDER BY LastName ASC, FirstName ASC;
5 GO

```

100 %

Results Messages

	LastName	FirstName
94	Zhang	Vincent
95	Zhang	Warren
96	Zhang	Wesley
97	Zhang	Willie
98	Zhang	Zachary
99	Zheng	Aimee
100	Zheng	Alan
101	Zheng	Alejandro
102	Zheng	Alisha
103	Zheng	Alvin
104	Zheng	Amy

Usando corchetes [^] (cualquier carácter individual que no esté dentro del rango especificado [a-t] o en conjunto [abc])

Como habrá adivinado, esto es lo contrario al uso anterior del operador SQL Like entre corchetes. Digamos que queremos obtener todos los registros donde el primer carácter de "Nombre" no comienza con [a a la f]:

```

SELECT BusinessEntityID, FirstName, LastName
FROM Person.Person
WHERE FirstName LIKE '[^a-f]';
GO

```

Observe que solo retornó los registros que no comienzan con ningún carácter de la "a" a la "f":

SQLQuery1.sql - D...TOP-9JU7UEF\boksi)*

```

1 SELECT BusinessEntityID, FirstName, LastName
2 FROM Person.Person
3 WHERE FirstName LIKE '[^a-f]%;';
4 GO

```

100 %

Results Messages

	BusinessEntityID	FirstName	LastName
1	285	Syed	Abbas
2	295	Kim	Abercrombie
3	2170	Kim	Abercrombie
4	38	Kim	Abercrombie
5	211	Hazem	Abolrous
6	2357	Sam	Abolrous
7	297	Humberto	Acevedo
8	291	Gustavo	Achong
9	299	Pilar	Ackerman
10	121	Pilar	Ackerman

Con el ejemplo del conjunto, supongamos que queremos obtener todos los registros en los cuales el "Nombre" no comienza con a, d, j. entonces podemos utilizar la siguiente consulta:

```

SELECT BusinessEntityID, FirstName, LastName
FROM Person.Person
WHERE FirstName LIKE '[^adj]%;';
GO

```

Aquí se muestra el conjunto de resultados:

SQLQuery1.sql - D...TOP-9JU7UEF\boksi)*

```

1 SELECT BusinessEntityID, FirstName, LastName
2 FROM Person.Person
3 WHERE FirstName LIKE '^adj)%';
4 GO

```

100 %

Results Messages

	BusinessEntityID	FirstName	LastName
1	285	Syed	Abbas
2	293	Catherine	Abel
3	295	Kim	Abercrombie
4	2170	Kim	Abercrombie
5	38	Kim	Abercrombie
6	211	Hazem	Abolrous
7	2357	Sam	Abolrous
8	297	Humberto	Acevedo
9	291	Gustavo	Achong
10	299	Pilar	Ackerman

Usando la sentencia de Escape

Este es un SQL Like similar al que se utiliza para especificar un carácter de escape.

La siguiente consulta utiliza la sentencia de escape y el carácter de escape:

USE tempdb;

GO

IF EXISTS(SELECT TABLE_NAME

FROM INFORMATION_SCHEMA.TABLES

WHERE TABLE_NAME = 'mytbl2')

DROP TABLE mytbl2;

GO

USE tempdb;

GO

CREATE TABLE mytbl2(c1 SYSNAME);

GO

INSERT INTO mytbl2

VALUES('Discount is 10-15% off'), ('Discount is .10-.15 off');

GO

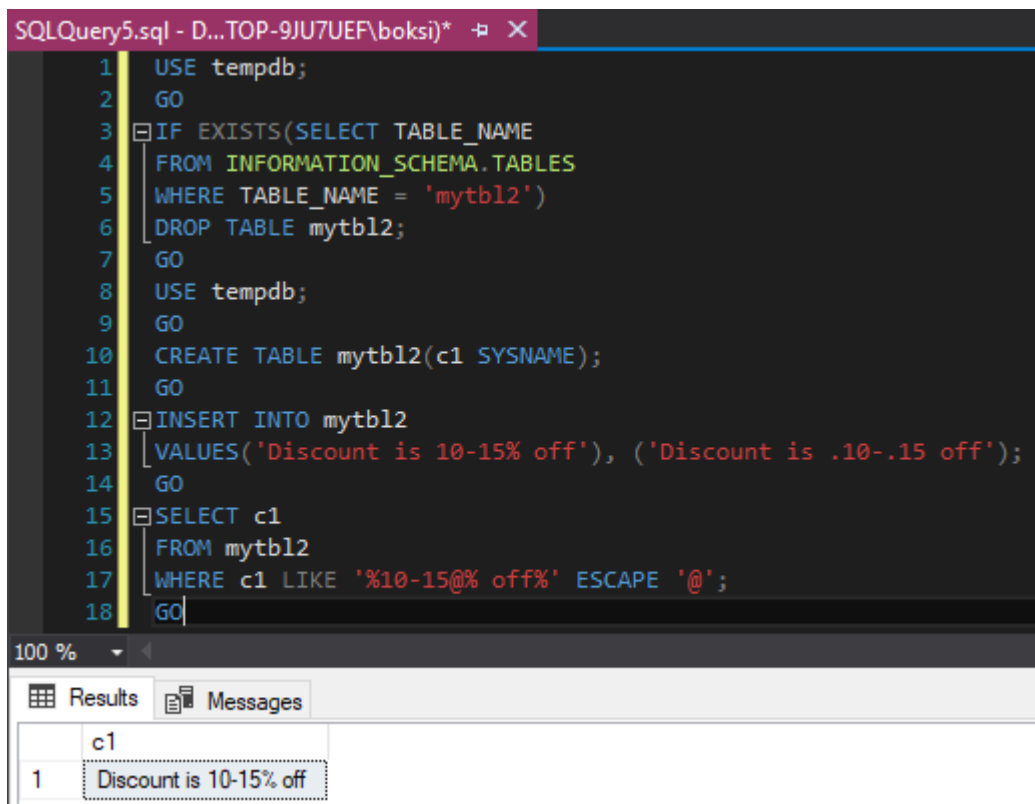
SELECT c1

FROM mytbl2

WHERE c1 LIKE '%10-15@% off%' ESCAPE '@';

GO

Devuelve la cadena de caracteres exacta de 10-15 % en la columna c1 de la tabla mytbl2:



The screenshot shows a SQL query window titled 'SQLQuery5.sql - D:\TOP-9JU7UEF\boksi)*'. The query is as follows:

```
1  USE tempdb;
2  GO
3  IF EXISTS(SELECT TABLE_NAME
4  FROM INFORMATION_SCHEMA.TABLES
5  WHERE TABLE_NAME = 'mytbl2')
6  DROP TABLE mytbl2;
7  GO
8  USE tempdb;
9  GO
10 CREATE TABLE mytbl2(c1 SYSNAME);
11 GO
12 INSERT INTO mytbl2
13 VALUES('Discount is 10-15% off'), ('Discount is .10-.15 off');
14 GO
15 SELECT c1
16 FROM mytbl2
17 WHERE c1 LIKE '%10-15@% off%' ESCAPE '@';
18 GO
```

Below the query window, the 'Results' tab is active, displaying a table with one row and one column:

	c1
1	Discount is 10-15% off

Espero realmente que este artículo sobre el operador SQL Like haya sido informativo y les agradezco mucho la lectura.

Operador NOT

os parámetros utilizados en la sintaxis mencionada anteriormente son los siguientes:

Columna_1, column_2, ...: Columnas o campos que deben mostrarse en el conjunto de resultados final.

Tabla: El nombre de la tabla de la que se deben obtener dichas columnas.

Condición: Condición en base a la cual se deben filtrar los registros. Más específicamente, la condición por la cual las filas deben devolver FALSO para que sea el conjunto de resultados final.

Habiendo discutido la sintaxis y el parámetro usado para escribir consultas SQL con NOT, probemos algunos ejemplos basados en él para comprender el concepto en detalle. Cuando tiene varias condiciones con el operador NOT, retornaran todas las filas que no cumplen las condiciones aplicadas. EL operador Not toma un único valor booleano como argumento y cambia su valor de falso a verdadero o de verdadero a falso....

Ejemplo operador sql not

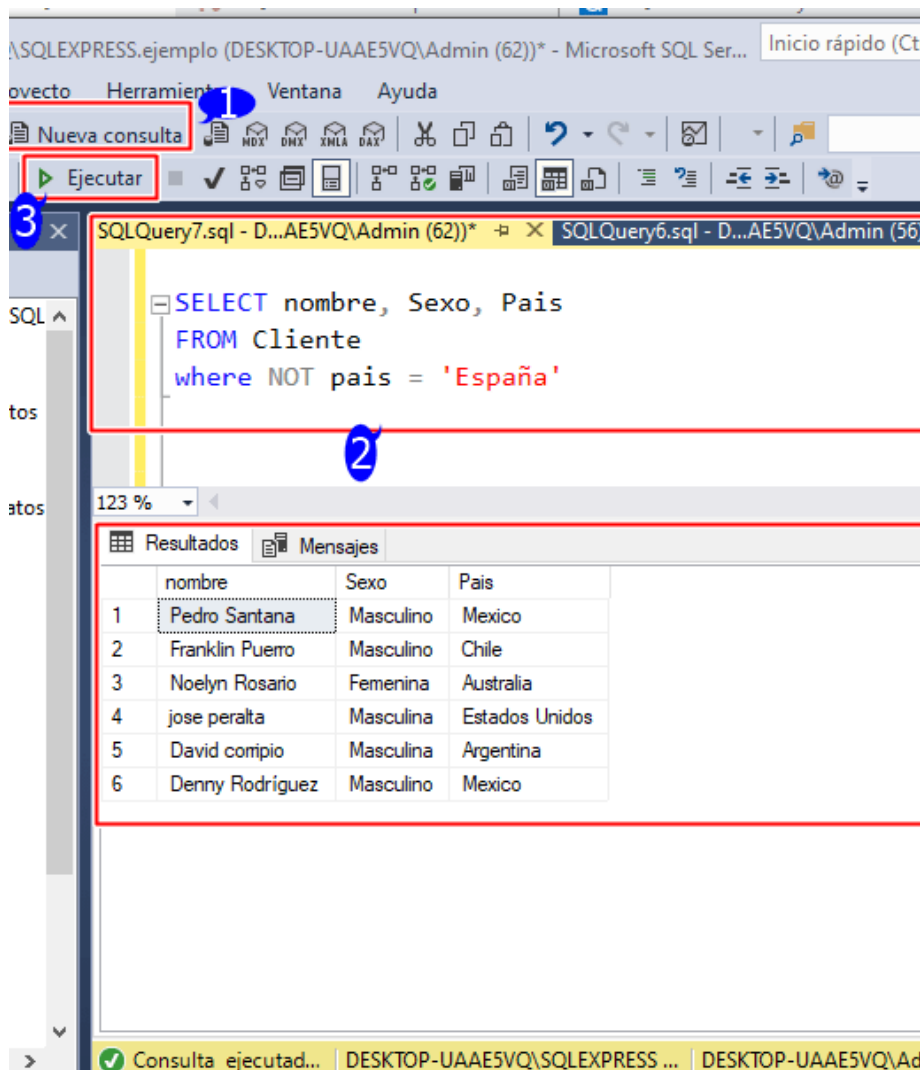
Podemos usar este operador con nuestro ejemplo con la tabla cliente. ¿Qué pasaría si quisiéramos encontrar a todos nuestros clientes que están fuera de España? Usando el operador sql NOT, simplemente podemos escribir:

```
SELECT nombre, Sexo, Pais
```

```
FROM Cliente
```

```
where NOT pais = 'España'
```

La ejecución de esta consulta en el management studio, da como resultado los valores en la siguiente imagen:



Puede ver claramente qué efecto ha tenido el operador NOT . No se devolvió a ningún miembro que tuviera su localización en España.

Conclusión: operador SQL NOT

El operador NOT se utiliza para filtrar registros en función de la negación de condiciones especificadas en la cláusula WHERE, HAVING, Not exists sql, not null sql y not like sql de una consulta SQL Server. Puntos importantes a tener en cuenta al utilizar el operador SQL «NOT». Aquí hay algunos puntos clave:

El operador «NOT» se utiliza para invertir una condición en una cláusula WHERE. es decir, en una condición «WHERE edad >= 25», el uso de «NOT» invertiría la condición a «WHERE NOT edad >= 25», lo que devolvería todos los registros en los que la edad es menor a 25.

El operador «NOT» también se puede utilizar junto con otros operadores como «AND» y «OR». Por ejemplo, si tienes una condición «WHERE edad >= 25 AND ciudad = 'Madrid'», el uso de «NOT» invertiría toda la condición a «WHERE NOT (edad >= 25 AND ciudad = 'Madrid')», lo que devolvería todos los registros en los que la edad es menor a 25 o la ciudad no es Madrid.

Es importante tener cuidado al utilizar el operador «NOT», ya que puede tener un impacto en el rendimiento de la consulta. Si utilizas «NOT» en una condición compleja, es posible que la base de datos tenga que realizar una búsqueda completa de la tabla para encontrar los registros que cumplan con la condición.

En algunas bases de datos, es posible que se requiera utilizar «NOT EXISTS» en lugar de «NOT» para consultar la ausencia de registros. Por ejemplo, si deseas buscar todos los clientes que no tienen pedidos en una tabla de pedidos, es posible que necesites utilizar una consulta como «SELECT * FROM clientes WHERE NOT EXISTS (SELECT * FROM pedidos WHERE pedidos.cliente_id = clientes.id)».

Operador BETWEEN

El operador BETWEEN se utiliza en la cláusula WHERE para seleccionar valores entre un rango de datos.

Sintaxis de SQL BETWEEN

SELECT columna

FROM tabla WHERE columna
BETWEEN valor1 AND valor2

Ejemplo de SQL BETWEEN

Dada la siguiente tabla 'personas'

nombre	apellido1	apellido2
ANTONIO	PEREZ	GOMEZ
ANTONIO	GARCIA	RODRIGUEZ
PEDRO	RUIZ	GONZALEZ

Seleccionar personas cuyo apellido1 esté entre 'FERNANDEZ y 'HUERTAS'

```
SELECT *  
FROM personas  
WHERE apellido1  
BETWEEN 'FERNANDEZ' AND 'HUERTAS'
```

nombre	apellido1	apellido2
ANTONIO	GARCIA	RODRIGUEZ

Seleccionar personas cuyo apellido1 no esté entre 'FERNANDEZ y 'HUERTAS'

```
SELECT *  
FROM personas  
WHERE apellido1  
NOT BETWEEN 'FERNANDEZ' AND 'HUERTAS'
```

nombre	apellido1	apellido2
ANTONIO	PEREZ	GOMEZ
PEDRO	RUIZ	GONZALEZ

Funciones de Texto

LEFT / RIGHT

Left, devuelve la parte izquierda de una cadena de caracteres con el número de caracteres especificado.

LEFT (character_expression , integer_expression)

Argumentos

character_expression

Es una expresión de datos binarios o de caracteres. character_expression puede ser una constante, una variable o una columna. character_expression puede ser cualquier tipo de datos (excepto text o ntext) que se pueda convertir implícitamente a varchar o nvarchar. De lo contrario, use la función CAST para convertir character_expression explícitamente.

Nota

Si string_expression es de tipo binary o varbinary, LEFT realizará una conversión implícita a varchar y, por tanto, no conservará la entrada binaria.

integer_expression

Es un entero positivo que especifica cuántos caracteres de character_expression se van a devolver. Si integer_expression es negativo, se devuelve un error. Si

`integer_expression` es de tipo `bigint` y contiene un valor grande, `character_expression` debe ser de un tipo de datos de gran tamaño, como `varchar(max)`.

El parámetro `integer_expression` cuenta un carácter suplente UTF 16 como un carácter.

Tipos de valor devuelto

Devuelve `varchar` cuando `character_expression` es de un tipo de datos de caracteres no Unicode.

Devuelve `nvarchar` cuando `character_expression` es de un tipo de datos de caracteres Unicode.

Observaciones

Al usar intercalaciones de SC, el parámetro `integer_expression` cuenta un par suplente UTF 16 como un carácter. Para más información, consulte [Compatibilidad con la intercalación y Unicode](#).

Ejemplos

A. Utilizar LEFT con una columna

En el ejemplo siguiente se devuelven los cinco caracteres situados más a la izquierda de cada nombre de producto de la tabla `Product` de la base de datos `AdventureWorks2022`.

```
SELECT LEFT(Name, 5)
FROM Production.Product
ORDER BY ProductID;
```

GO

B. Utilizar LEFT con una cadena de caracteres

En el ejemplo siguiente se utiliza LEFT para devolver los dos caracteres situados más a la izquierda de la cadena de caracteres abcdefg.

```
SELECT LEFT('abcdefg',2);
```

GO

El conjunto de resultados es el siguiente:

--

ab

(1 row(s) affected)

C. Utilizar LEFT con una columna

En el ejemplo siguiente se devuelven los cinco caracteres situados más a la izquierda de cada nombre de producto.

-- Uses AdventureWorks

```
SELECT LEFT(EnglishProductName, 5)
```

```
FROM dbo.DimProduct
```

```
ORDER BY ProductKey;
```

D. Utilizar LEFT con una cadena de caracteres

En el ejemplo siguiente se utiliza LEFT para devolver los dos caracteres situados más a la izquierda de la cadena de caracteres abcdefg.

-- Uses AdventureWorks

```
SELECT LEFT('abcdefg',2) FROM dbo.DimProduct;
```

El conjunto de resultados es el siguiente:

--

ab

Right, devuelve la parte derecha de una cadena de caracteres con el número de caracteres especificado.

RIGHT (character_expression , integer_expression)

Argumentos

character_expression

Es una expresión de datos binarios o de caracteres. character_expression puede ser una constante, una variable o una columna. character_expression puede ser cualquier tipo de datos (excepto text o ntext) que se pueda convertir implícitamente a varchar o nvarchar. De lo contrario, use la función CAST para convertir character_expression explícitamente.

Nota

Si string_expression es de tipo binary o varbinary, RIGHT realizará una conversión implícita a varchar y, por tanto, no conservará la entrada binaria.

integer_expression

Es un entero positivo que especifica cuántos caracteres de character_expression se van a devolver. Si integer_expression es negativo, se devuelve un error. Si integer_expression es de tipo bigint y contiene un valor grande, character_expression debe ser de un tipo de datos de gran tamaño, como varchar(max) .

Tipos de valor devuelto

Devuelve varchar cuando character_expression es de un tipo de datos de caracteres no Unicode.

Devuelve nvarchar cuando character_expression es de un tipo de datos de caracteres Unicode.

Caracteres adicionales (pares suplentes)

Al utilizar las intercalaciones de SC, la función RIGHT cuenta un par suplente UTF 16 como un carácter individual. Para más información, consulte Compatibilidad con la intercalación y Unicode.

Ejemplos

A. Usar RIGHT con una columna

En el ejemplo siguiente se devuelven los cinco caracteres situados más a la derecha del nombre de cada persona de la base de datos AdventureWorks2022.

```
SELECT RIGHT(FirstName, 5) AS 'First Name'  
FROM Person.Person  
WHERE BusinessEntityID < 5
```

```
ORDER BY FirstName;
```

```
GO
```

El conjunto de resultados es el siguiente:

```
First Name
```

```
-----
```

```
Ken
```

```
Terri
```

```
berto
```

```
Rob
```

(4 row(s) affected)

B. Usar RIGHT con una columna

En el siguiente ejemplo se devuelven los cinco caracteres situados más a la derecha de cada apellido de la tabla DimEmployee.

```
-- Uses AdventureWorks
```

```
SELECT RIGHT(LastName, 5) AS Name
```

```
FROM dbo.DimEmployee
```

```
ORDER BY EmployeeKey;
```

A continuación se muestra un conjunto parcial de resultados.

```
Name
```

```
-----
```


lbert

Brown

relo

lbers

C. Usar RIGHT con una cadena de caracteres

En el siguiente ejemplo se usa RIGHT para devolver los dos caracteres situados más a la derecha de la cadena de caracteres abcdefg.

```
SELECT RIGHT('abcdefg', 2);
```

El conjunto de resultados es el siguiente:

fg

LEN

Devuelve el número de caracteres de la expresión de cadena especificada, excluidos los espacios finales.

Sintaxis

LEN (string_expression)

Argumentos

string_expression

Es la expresión de cadena que se va a evaluar. string_expression puede ser una constante, una variable o una columna de datos binarios o de caracteres.

Tipos de valor devuelto

bigint si expression es de los tipos de datos varchar(max) , nvarchar(max) o varbinary(max) ; en caso contrario, es int.

Si utiliza intercalaciones de SC, el valor entero devuelto cuenta los pares suplentes UTF-16 como un solo carácter. Para más información, consulte Compatibilidad con la intercalación y Unicode.

Comentarios

LEN no incluye espacios finales. Si esto supone un problema, considere la opción de usar la función DATALENGTH (Transact-SQL), que no recorta la cadena. Si se procesa una cadena unicode, DATALENGTH devolverá un número que tal vez no sea igual al número de caracteres. En este ejemplo se muestra LEN y DATALENGTH con un espacio final.

```
DECLARE @v1 VARCHAR(40),
        @v2 NVARCHAR(40);
SELECT
    @v1 = 'Test of 22 characters ',
    @v2 = 'Test of 22 characters ';
SELECT LEN(@v1) AS [VARCHAR LEN] , DATALENGTH(@v1) AS [VARCHAR
DATALENGTH];
SELECT LEN(@v2) AS [NVARCHAR LEN], DATALENGTH(@v2) AS [NVARCHAR
DATALENGTH];
```

Nota

Use LEN para devolver el número de caracteres codificados en una expresión de cadena determinada y DATALENGTH para devolver el tamaño en bytes para una

expresión de cadena determinada. Estos resultados pueden diferir en función del tipo de datos y del tipo de codificación utilizado en la columna. Para obtener más información sobre las diferencias de almacenamiento entre los distintos tipos de codificación, consulte Compatibilidad con la intercalación y Unicode.

Ejemplos

El ejemplo siguiente selecciona el número de caracteres y los datos en FirstName para las personas que se encuentran en Australia. En este ejemplo se utiliza la base de datos de AdventureWorks.

```
SELECT LEN(FirstName) AS Length, FirstName, LastName
FROM Sales.vIndividualCustomer
WHERE CountryRegionName = 'Australia';
GO
```

En este ejemplo se devuelve el número de caracteres en la columna FirstName y los nombres y apellidos de los empleados que se encuentran en Australia.

```
USE AdventureWorks2022
GO
SELECT DISTINCT LEN(FirstName) AS FNameLength, FirstName, LastName
FROM dbo.DimEmployee AS e
INNER JOIN dbo.DimGeography AS g
    ON e.SalesTerritoryKey = g.SalesTerritoryKey
WHERE EnglishCountryRegionName = 'Australia';
```

El conjunto de resultados es el siguiente:

FNameLength	FirstName	LastName
-------------	-----------	----------

LOWER Y UPPER

LOWER. Devuelve una expresión de caracteres después de convertir en minúsculas los datos de caracteres en mayúsculas.

Sintaxis

LOWER (character_expression)

Argumentos

character_expression

Es una expresión de datos binarios o de caracteres. character_expression puede ser una constante, una variable o una columna. character_expression debe ser de un tipo de datos que se pueda convertir implícitamente a varchar. De lo contrario, use CAST para convertir character_expression explícitamente.

Tipos de valor devueltos

varchar o nvarchar

Ejemplos

En este ejemplo se utiliza la función LOWER y la función UPPER, y se anida la función UPPER en la función LOWER al seleccionar títulos de libros cuyos precios sean entre 11\$ y 20\$.

-- Uses AdventureWorks

```
SELECT LOWER(SUBSTRING(EnglishProductName, 1, 20)) AS Lower,
```

```

UPPER(SUBSTRING(EnglishProductName, 1, 20)) AS Upper,
LOWER(UPPER(SUBSTRING(EnglishProductName, 1, 20))) As LowerUpper
FROM dbo.DimProduct
WHERE ListPrice between 11.00 and 20.00;

```

El conjunto de resultados es el siguiente:

Lower	Upper	LowerUpper
minipump	MINIPUMP	minipump
taillights - battery	TAILLIGHTS - BATTERY	taillights – battery

UPPER. Devuelve una expresión de caracteres con datos de caracteres en minúsculas convertidos a mayúsculas.

Sintaxis

UPPER (character_expression)

Argumentos

character_expression

Es una expresión de datos de caracteres. character_expression puede ser una constante, una variable o una columna de datos binarios o de caracteres.

character_expression debe ser de un tipo de datos que se pueda convertir implícitamente a varchar. De lo contrario, use CAST para convertir character_expression explícitamente.

Tipos de valor devueltos

varchar o nvarchar

Ejemplos

En el ejemplo siguiente se utilizan las funciones UPPER y RTRIM para devolver el apellido de las personas de la tabla dbo.DimEmployee de manera que aparezca en mayúsculas, recortado y concatenado al nombre.

-- Uses AdventureWorks

```
SELECT UPPER(RTRIM(LastName)) + ', ' + FirstName AS Name  
FROM dbo.DimEmployee  
ORDER BY LastName;
```

A continuación se muestra un conjunto parcial de resultados.

Name

ABBAS, Syed

ABERCROMBIE, Kim

ABOLROUS, Hazem

REPLICATE

Repite un valor de cadena un número especificado de veces.

Sintaxis

REPLICATE (string_expression , integer_expression)

Argumentos

string_expression

Es una expresión de un tipo de datos binario o de cadena de caracteres.

Nota

Si string_expression es de tipo binary, REPLICATE realizará una conversión implícita a varchar y, por tanto, no conservará la entrada binaria.

Nota

Si la entrada string_expression no es de tipo varchar(max) o nvarchar(max), REPLICATE trunca el valor devuelto en 8000 bytes. Para devolver valores mayores de 8000 bytes, string_expression debe convertirse explícitamente al tipo de datos de valores grandes apropiado.

integer_expression

Es una expresión de cualquier tipo entero, incluido bigint. Si integer_expression es negativo, se devuelve NULL.

Tipos de valor devuelto

Devuelve el mismo tipo que string_expression.

Ejemplos

A. Usar REPLICATE

En el ejemplo siguiente se replica un carácter 0 cuatro veces delante de un código de línea de producción en la base de datos AdventureWorks2022.

```

SELECT [Name]
, REPLICATE('0', 4) + [ProductLine] AS 'Line Code'
FROM [Production].[Product]
WHERE [ProductLine] = 'T'
ORDER BY [Name];
GO

```

El conjunto de resultados es el siguiente:

Name	Line Code
HL Touring Frame - Blue, 46	0000T
HL Touring Frame - Blue, 50	0000T
HL Touring Frame - Blue, 54	0000T
HL Touring Frame - Blue, 60	0000T
HL Touring Frame - Yellow, 46	0000T
HL Touring Frame - Yellow, 50	0000T
...	

B. Usar REPLICATE y DATALENGTH

En el ejemplo siguiente se rellena de números a la izquierda hasta una longitud especificada mientras que los números se convierten de un tipo de datos numérico a caracteres o Unicode.

```

IF EXISTS(SELECT name FROM sys.tables
WHERE name = 't1')
DROP TABLE t1;
GO
CREATE TABLE t1

```



```
(
  c1 varchar(3),
  c2 char(3)
);
GO
INSERT INTO t1 VALUES ('2', '2'), ('37', '37'),('597', '597');
GO
SELECT REPLICATE('0', 3 - DATALENGTH(c1)) + c1 AS 'Varchar Column',
       REPLICATE('0', 3 - DATALENGTH(c2)) + c2 AS 'Char Column'
FROM t1;
GO
```

El conjunto de resultados es el siguiente:

Varchar Column	Char Column
002	2
037	37
597	597

(3 row(s) affected)

C. Usar REPLICATE

En el siguiente ejemplo se replica un carácter 0 cuatro veces delante de un valor ItemCode.

```
-- Uses AdventureWorks
```

```

SELECT EnglishProductName AS Name,
       ProductAlternateKey AS ItemCode,
       REPLICATE('0', 4) + ProductAlternateKey AS FullItemCode
FROM dbo.DimProduct
ORDER BY Name;

```

Estas son las primeras filas del conjunto de resultados.

Name	ItemCode	FullItemCode
-----	-----	-----
Adjustable Race	AR-5381	0000AR-5381
All-Purpose Bike Stand	ST-1401	0000ST-1401
AWC Logo Cap	CA-1098	0000CA-1098
AWC Logo Cap	CA-1098	0000CA-1098
AWC Logo Cap	CA-1098	0000CA-1098
BB Ball Bearing	BE-2349	0000BE-2349

LTRIM Y RTRIM

LTRIM. Quita el carácter de espacio char(32) u otros caracteres especificados del final de una cadena.

Sintaxis

LTRIM (character_expression)

Importante

Necesitará el nivel de compatibilidad de la base de datos establecido en 160 para usar el argumento de caracteres opcional.

`LTRIM ( character_expression , [ characters ] )`

Argumentos

`character_expression`

Una expresión de datos binarios o de caracteres. `character_expression` puede ser una constante, una variable o una columna. `character_expression` debe ser de un tipo de datos (excepto `text`, `ntext` e `image`) que se pueda convertir implícitamente a `varchar`. De lo contrario, use `CAST` para convertir `character_expression` explícitamente.

`characters`

Un literal, una variable o una llamada de función de cualquier tipo de carácter no de LOB (`nvarchar`, `varchar`, `nchar`, o `char`) que contiene caracteres que se deben quitar. No se permiten los tipos `nvarchar(max)` y `varchar(max)`.

Tipos de valores devueltos

Devuelve una expresión de caracteres con un tipo de argumento de cadena donde el carácter de espacio `char(32)` u otros caracteres especificados se quitan del principio de un `character_expression`. Devuelve `NULL` si la cadena de entrada es `NULL`.

Comentarios

Para habilitar los argumentos posicionales de caracteres, debe habilitar el nivel 160 de compatibilidad de la base de datos en la o las bases de datos a las que se va a conectar al ejecutar consultas.

Ejemplos

A. Quitar espacios iniciales

En el siguiente ejemplo se usa LTRIM para quitar los espacios iniciales de una expresión de caracteres.

```
SELECT LTRIM('    Five spaces are at the beginning of this string.');
```

El conjunto de resultados es el siguiente:

Five spaces are at the beginning of this string.

B: Quitar espacios iniciales mediante una variable

En el siguiente ejemplo se utiliza LTRIM para quitar espacios iniciales de una variable de caracteres.

```
DECLARE @string_to_trim VARCHAR(60);  
SET @string_to_trim = '    Five spaces are at the beginning of this string.';  
SELECT  
    @string_to_trim AS 'Original string',  
    LTRIM(@string_to_trim) AS 'Without spaces';  
GO
```

El conjunto de resultados es el siguiente:

Original string

Without spaces

Five spaces are at the beginning of this string. Five spaces are at the beginning of this string.

Quitar caracteres especificados del principio y el final de una cadena

Importante

Necesitará el nivel de compatibilidad de la base de datos establecido en 160 para usar el argumento de caracteres opcional.

En el ejemplo siguiente se quitan los caracteres 123 del principio de la cadena 123abc..

```
SELECT LTRIM('123abc.' , '123.');
```

El conjunto de resultados es el siguiente:

abc.

RTRIM. Quita el carácter de espacio char(32) u otros caracteres especificados del final de una cadena.

Sintaxis

RTRIM (character_expression)

RTRIM (character_expression , [characters])

Argumentos

character_expression

Una expresión de datos binarios o de caracteres. character_expression puede ser una constante, una variable o una columna. character_expression debe ser de un tipo de datos (excepto text, ntext e image) que se pueda convertir implícitamente a varchar. De lo contrario, use CAST para convertir character_expression explícitamente.

characters

Se aplica a: SQL Server 2022 (16.x) y versiones posteriores.

Un literal, una variable o una llamada de función de cualquier tipo de carácter no de LOB (nvarchar, , varchar, nchar, o char) que contiene caracteres que se deben quitar. No se permiten los tipos nvarchar(max) y varchar(max).

Tipos de valores devueltos

Devuelve una expresión de caracteres con un tipo de argumento de cadena donde el carácter de espacio char(32) u otros caracteres especificados se quitan del final de un character_expression. Devuelve NULL si la cadena de entrada es NULL.

Comentarios

Para habilitar los argumentos posicionales de caracteres, debe habilitar el nivel 160 de compatibilidad de la base de datos en la o las bases de datos a las que se va a conectar al ejecutar consultas.

Ejemplos

A. Quitar los espacios finales.

El siguiente ejemplo toma una cadena de caracteres que tiene espacios al final de la frase y devuelve el texto sin esos espacios.

```
SELECT RTRIM('Removes trailing spaces. ');
```

El conjunto de resultados es el siguiente:

Removes trailing spaces.

B. Quitar espacios finales con una variable

En el ejemplo siguiente se muestra cómo utilizar RTRIM para quitar los espacios finales de una variable de caracteres.

```
DECLARE @string_to_trim VARCHAR(60);  
SET @string_to_trim = 'Four spaces are after the period in this sentence.   ';  
SELECT @string_to_trim + ' Next string.';  
SELECT RTRIM(@string_to_trim) + ' Next string.';  
GO
```

El conjunto de resultados es el siguiente:

Four spaces are after the period in this sentence. Next string.

Four spaces are after the period in this sentence. Next string.

C. Eliminación de los caracteres especificados de ambos lados de la cadena

En el ejemplo siguiente se quitan los caracteres abc. del final de la cadena .123abc..

```
SELECT RTRIM('.123abc.' , 'abc.');
```

El conjunto de resultados es el siguiente:

.123

CONCAT

Esta función devuelve una cadena resultante de la concatenación, o la combinación, de dos o más valores de cadena de una manera integral. (Para agregar un valor de separación durante la concatenación, vea [CONCAT_WS](#)).

Sintaxis

```
CONCAT ( string_value1, string_value2 [, string_valueN ] )
```

Argumentos

string_value

Valor de cadena que se va a concatenar con los demás valores. La función CONCAT requiere al menos dos argumentos valor_cadena y no más de 254 argumentos valor_cadena.

Tipos de valores devueltos

string_value

Un valor de cadena cuya longitud y tipo dependen de la entrada.

Observaciones

CONCAT toma un número variable de argumentos de cadena y los concatena (o combina) en una sola cadena. Necesita un mínimo de dos valores de entrada; de lo contrario, se produce un error en CONCAT. CONCAT convierte implícitamente todos los argumentos en tipos de cadena antes de la concatenación. CONCAT convierte implícitamente los valores NULL en cadenas vacías. Si CONCAT recibe argumentos en los que todos los valores son NULL, devolverá una cadena vacía de tipo varchar(1). La conversión implícita de cadenas sigue las reglas existentes para las conversiones de tipos de datos. Para obtener más información sobre las conversiones de tipo de datos, consulte CAST y CONVERT (Transact-SQL).

El tipo devuelto depende del tipo de los argumentos. En esta tabla se muestra la asignación:

Argumentos

string_value

Valor de cadena que se va a concatenar con los demás valores. La función CONCAT requiere al menos dos argumentos *valor_cadena* y no más de 254 argumentos *valor_cadena*.

Tipos de valores devueltos

string_value

Un valor de cadena cuya longitud y tipo dependen de la entrada.

Observaciones

CONCAT toma un número variable de argumentos de cadena y los concatena (o combina) en una sola cadena. Necesita un mínimo de dos valores de entrada; de lo contrario, se produce un error en CONCAT. CONCAT convierte implícitamente todos los argumentos en tipos de cadena antes de la concatenación. CONCAT convierte implícitamente los valores NULL en cadenas vacías. Si CONCAT recibe argumentos en los que todos los valores son **NULL**, devolverá una cadena vacía de tipo **varchar**(1). La conversión implícita de cadenas sigue las reglas existentes para

las conversiones de tipos de datos. Para obtener más información sobre las conversiones de tipo de datos, consulte [CAST y CONVERT \(Transact-SQL\)](#).

El tipo devuelto depende del tipo de los argumentos. En esta tabla se muestra la asignación:

Cuando CONCAT recibe argumentos de entrada nvarchar de longitud ≤ 4000 caracteres, o bien argumentos de entrada varchar de longitud ≤ 8000 caracteres, las conversiones implícitas pueden afectar a la longitud del resultado. Otros tipos de datos tienen otras longitudes cuando se convierten implícitamente a cadenas. Por ejemplo, un tipo int (14) tiene una longitud de cadena de 12, mientras que un tipo float tiene una longitud de 32. Por tanto, una concatenación de dos enteros devuelve un resultado con una longitud de no menos de 24.

Si ninguno de los argumentos de entrada tiene un tipo de objeto grande (LOB) admitido, el tipo devuelto se trunca a una longitud de 8 000 caracteres, independientemente del tipo de valor devuelto. Este truncamiento ahorra espacio y admite la eficacia de la generación de planes.

La función CONCAT se puede ejecutar de forma remota en un servidor vinculado de la versión SQL Server 2012 (11.x) o versiones posteriores. Para servidores vinculados anteriores, la operación de CONCAT se realizará localmente, después de que el servidor vinculado devuelva los valores no concatenados.

Ejemplos

A. Usar CONCAT

```
SELECT CONCAT ( 'Happy ', 'Birthday ', 11, '/', '25' ) AS Result;
```

El conjunto de resultados es el siguiente:

Result

Happy Birthday 11/25

(1 row(s) affected)

B. Usar CONCAT con valores NULL

```
CREATE TABLE #temp (  
    emp_name NVARCHAR(200) NOT NULL,  
    emp_middlename NVARCHAR(200) NULL,  
    emp_lastname NVARCHAR(200) NOT NULL  
);  
INSERT INTO #temp VALUES( 'Name', NULL, 'Lastname' );  
SELECT CONCAT( emp_name, emp_middlename, emp_lastname ) AS Result  
FROM #temp;
```

El conjunto de resultados es el siguiente:

Result

NameLastname

(1 row(s) affected)

Funciones de Fecha

DATEADD

Esta función agrega un elemento number (un entero con signo) a un elemento datepart de un elemento date de entrada y devuelve un valor de fecha y hora modificado. Por ejemplo, puede usar esta función para buscar la fecha que es 7000 minutos a partir de hoy: number = 7000, datepart = minute, date = today.

Para obtener una introducción sobre todos los tipos de datos y funciones de fecha y hora de Transact-SQL, vea Tipos de datos y [funciones de fecha y hora](#) (Transact-SQL).

Sintaxis

DATEADD (datepart , number , date)

Argumentos

datepart

La parte de la fecha a la que DATEADD agrega un número entero. En esta tabla se enumeran todos los argumentos válidos de datepart.

Nota

DATEADD no acepta los equivalentes de variables definidas por el usuario para los argumentos datepart.

<i>datepart</i>	Abreviaturas
year	YY, YYYY

quarter	qq, q
month	mm, m
dayofyear	dy, y
day	dd, d
week	wk, ww
weekday	dw, w
hour	hh
minute	mi, n
second	ss, s
millisecond	ms
microsecond	mcs
nanosecond	ns

número

Expresión que se puede resolver como un valor int que DATEADD suma a un atributo datepart de date. DATEADD acepta valores de variables definidas por el usuario para number. DATEADD truncará un valor number especificado que tiene una fracción decimal. En esta situación no se redondeará el valor number.

date

Una expresión que se puede resolver en uno de los valores siguientes:

date

datetime

datetimeoffset

datetime2

smalldatetime

time

Para date, DATEADD aceptará una expresión de columna, una expresión, un literal de cadena o una variable definida por el usuario. Un valor de literal de cadena se debe resolver en un argumento datetime. Para evitar problemas de ambigüedad, use años de cuatro dígitos. Vea Establecer la opción de configuración del servidor Fecha límite de año de dos dígitos para obtener información sobre los años de dos dígitos.

Tipos de valores devueltos

El tipo de datos de valores devueltos de este método es dinámico. El tipo devuelto depende del argumento proporcionado para date. Si el valor de date es una fecha de literal de cadena, DATEADD devuelve un valor datetime. Si se proporciona otro tipo válido de datos de entrada para date, DATEADD devuelve el mismo tipo de datos. DATEADD produce un error si la escala de segundos del literal de cadena supera tres posiciones decimales (.nnn) o si el literal de cadena contiene la parte de desplazamiento de zona horaria.

Valor devuelto

Argumento datepart

dayofyear, day y weekday devuelven el mismo valor.

Cada datepart y sus abreviaturas devuelven el mismo valor.

Si se cumple lo siguiente:

datepart es month

el mes de date tiene más días que el mes que se devuelve

el día de date no existe en el mes que se devuelve

Después, DATEADD devuelve el último día del mes que se devuelve. Por ejemplo, septiembre tiene 30 días; por tanto, estas instrucciones devuelven 30-09-2006 00:00:00.000:

```
SELECT DATEADD(month, 1, '20060830');  
SELECT DATEADD(month, 1, '2006-08-31');
```

Argumento number

El argumento number no puede superar el intervalo de int. En estas instrucciones, el argumento para number supera el intervalo de int en uno. Estas instrucciones devuelven el mensaje de error siguiente: "Msg 8115, Level 16, State 2, Line 1. Arithmetic overflow error converting expression to data type int."

```
SELECT DATEADD(year,2147483648, '20060731');  
SELECT DATEADD(year,-2147483649, '20060731');
```

Argumento date

DATEADD no aceptará un argumento date incrementado a un valor fuera del intervalo de su tipo de datos. En las instrucciones siguientes, el valor number que se agrega al valor date supera el intervalo del tipo de datos date. DATEADD devuelve el mensaje de error siguiente: "Msg 517, Level 16, State 1, Line 1 Adding a value to a 'datetime' column caused overflow".

```
SELECT DATEADD(year,2147483647, '20060731');  
SELECT DATEADD(year,-2147483647, '20060731');
```

Valores devueltos para una fecha smalldatetime y datepart de un segundo o fracciones de segundo

La parte correspondiente a los segundos de un valor `smalldatetime` siempre es 00.
Para un valor de `fechasmalldatetime`, se aplica lo siguiente:

Para un argumento `datepart` de `second` y un valor `number` comprendido entre -30 y +29, `DATEADD` no realiza ningún cambio.

Para un argumento `datepart` de `second` y un valor `number` menor que -30 o mayor de +29, `DATEADD` realiza la suma a partir de un minuto.

Para un argumento `datepart` de `millisecond` y un valor `number` comprendido entre -30001 y +29998, `DATEADD` no realiza ningún cambio.

Para un argumento `datepart` de `millisecond` y un valor `number` menor que -30001 o mayor de +29998, `DATEADD` realiza la suma a partir de un minuto.

Observaciones

Use `DATEADD` en las cláusulas siguientes:

`GROUP BY`

`HAVING`

`ORDER BY`

`SELECT <lista>`

`WHERE`

Precisión de fracciones de segundo

`DATEADD` no permite sumar un valor `datepart` de `microsecond` o `nanosecond` para los tipos de datos de `date` como `smalldatetime`, `date` y `datetime`.

Los milisegundos tienen una escala de 3 (0,123), los microsegundos tienen una escala de 6 (0,123456) y los nanosegundos tienen una escala de 9 (0,123456789). Los tipos de datos `time`, `datetime2` y `datetimeoffset` tienen una escala máxima de 7 (.1234567). Para un argumento `datepart` de `nanosecond`, `number` debe ser 100 antes de que aumenten las fracciones de segundos de `date`. Un valor `number`

comprendido entre 1 y 49 se redondeará hacia abajo hasta 0, y un número entre 50 a 99 se redondeará hacia arriba hasta 100.

Estas instrucciones agregan un atributo datepart de millisecond, microsecond o nanosecond.

```
DECLARE @datetime2 datetime2 = '2007-01-01 13:10:10.1111111';
SELECT '1 millisecond', DATEADD(millisecond,1,@datetime2)
UNION ALL
SELECT '2 milliseconds', DATEADD(millisecond,2,@datetime2)
UNION ALL
SELECT '1 microsecond', DATEADD(microsecond,1,@datetime2)
UNION ALL
SELECT '2 microseconds', DATEADD(microsecond,2,@datetime2)
UNION ALL
SELECT '49 nanoseconds', DATEADD(nanosecond,49,@datetime2)
UNION ALL
SELECT '50 nanoseconds', DATEADD(nanosecond,50,@datetime2)
UNION ALL
SELECT '150 nanoseconds', DATEADD(nanosecond,150,@datetime2);
```

El conjunto de resultados es el siguiente:

1 millisecond	2007-01-01 13:10:10.1121111
2 milliseconds	2007-01-01 13:10:10.1131111
1 microsecond	2007-01-01 13:10:10.1111121
2 microseconds	2007-01-01 13:10:10.1111131

49 nanoseconds 2007-01-01 13:10:10.1111111
50 nanoseconds 2007-01-01 13:10:10.1111112
150 nanoseconds 2007-01-01 13:10:10.1111113

Ajuste de zona horaria

DATEADD no permite la suma para el desplazamiento de zona horaria.

Ejemplos

A. Aumentar datepart en un intervalo de 1

Cada una de estas instrucciones aumenta datepart en un intervalo de 1:

```
DECLARE @datetime2 datetime2 = '2007-01-01 13:10:10.1111111';
```

```
SELECT 'year', DATEADD(year,1,@datetime2)
```

```
UNION ALL
```

```
SELECT 'quarter',DATEADD(quarter,1,@datetime2)
```

```
UNION ALL
```

```
SELECT 'month',DATEADD(month,1,@datetime2)
```

```
UNION ALL
```

```
SELECT 'dayofyear',DATEADD(dayofyear,1,@datetime2)
```

```
UNION ALL
```

```
SELECT 'day',DATEADD(day,1,@datetime2)
```

```
UNION ALL
```

```
SELECT 'week',DATEADD(week,1,@datetime2)
```

```
UNION ALL
```

```
SELECT 'weekday',DATEADD(weekday,1,@datetime2)
```

```
UNION ALL
```

```
SELECT 'hour',DATEADD(hour,1,@datetime2)
```

UNION ALL

SELECT 'minute',DATEADD(minute,1,@datetime2)

UNION ALL

SELECT 'second',DATEADD(second,1,@datetime2)

UNION ALL

SELECT 'millisecond',DATEADD(millisecond,1,@datetime2)

UNION ALL

SELECT 'microsecond',DATEADD(microsecond,1,@datetime2)

UNION ALL

SELECT 'nanosecond',DATEADD(nanosecond,1,@datetime2);

El conjunto de resultados es el siguiente:

Year	2008-01-01 13:10:10.1111111
quarter	2007-04-01 13:10:10.1111111
month	2007-02-01 13:10:10.1111111
dayofyear	2007-01-02 13:10:10.1111111
day	2007-01-02 13:10:10.1111111
week	2007-01-08 13:10:10.1111111
weekday	2007-01-02 13:10:10.1111111
hour	2007-01-01 14:10:10.1111111
minute	2007-01-01 13:11:10.1111111
second	2007-01-01 13:10:11.1111111
millisecond	2007-01-01 13:10:10.1121111
microsecond	2007-01-01 13:10:10.1111121
nanosecond	2007-01-01 13:10:10.1111111

B. Aumentar más de un nivel de datepart en una instrucción

Cada una de estas instrucciones aumenta datepart en un valor number lo suficientemente grande como para incrementar también el siguiente valor superior datepart de date:

```
DECLARE @datetime2 datetime2;
```

```
SET @datetime2 = '2007-01-01 01:01:01.1111111';
```

```
--Statement
```

```
Result
```

```
-----  
SELECT DATEADD(quarter,4,@datetime2); --2008-01-01 01:01:01.1111111  
SELECT DATEADD(month,13,@datetime2); --2008-02-01 01:01:01.1111111  
SELECT DATEADD(dayofyear,365,@datetime2); --2008-01-01 01:01:01.1111111  
SELECT DATEADD(day,365,@datetime2); --2008-01-01 01:01:01.1111111  
SELECT DATEADD(week,5,@datetime2); --2007-02-05 01:01:01.1111111  
SELECT DATEADD(weekday,31,@datetime2); --2007-02-01 01:01:01.1111111  
SELECT DATEADD(hour,23,@datetime2); --2007-01-02 00:01:01.1111111  
SELECT DATEADD(minute,59,@datetime2); --2007-01-01 02:00:01.1111111  
SELECT DATEADD(second,59,@datetime2); --2007-01-01 01:02:00.1111111  
SELECT DATEADD(millisecond,1,@datetime2); --2007-01-01 01:01:01.1121111
```

C. Utilizar las expresiones como argumentos para los parámetros number y date

En los ejemplos siguientes se usan otros tipos de expresiones como argumentos para los parámetros number y date. En los ejemplos se usa la base de datos de AdventureWorks.

Especificar una columna como fecha

En este ejemplo se agregan 2 (dos) días a cada valor de la columna OrderDate para derivar una nueva columna denominada PromisedShipDate:

```

SELECT SalesOrderID
       ,OrderDate
       ,DATEADD(day,2,OrderDate) AS PromisedShipDate
FROM Sales.SalesOrderHeader;

```

Un conjunto de resultados parcial:

SalesOrderID	OrderDate	PromisedShipDate
-----	-----	-----
43659	2005-07-01 00:00:00.000	2005-07-03 00:00:00.000
43660	2005-07-01 00:00:00.000	2005-07-03 00:00:00.000
43661	2005-07-01 00:00:00.000	2005-07-03 00:00:00.000
...		
43702	2005-07-02 00:00:00.000	2005-07-04 00:00:00.000
43703	2005-07-02 00:00:00.000	2005-07-04 00:00:00.000
43704	2005-07-02 00:00:00.000	2005-07-04 00:00:00.000
43705	2005-07-02 00:00:00.000	2005-07-04 00:00:00.000
43706	2005-07-03 00:00:00.000	2005-07-05 00:00:00.000
...		
43711	2005-07-04 00:00:00.000	2005-07-06 00:00:00.000
43712	2005-07-04 00:00:00.000	2005-07-06 00:00:00.000
...		
43740	2005-07-11 00:00:00.000	2005-07-13 00:00:00.000
43741	2005-07-12 00:00:00.000	2005-07-14 00:00:00.000

Especificar las variables definidas por el usuario como number y date

En este ejemplo se especifican variables definidas por el usuario como argumentos para number y date:

```
DECLARE @days INT = 365,  
        @datetime DATETIME = '2000-01-01 01:01:01.111'; /* 2000 was a leap year  
*/;  
SELECT DATEADD(day, @days, @datetime);
```

El conjunto de resultados es el siguiente:

```
-----  
2000-12-31 01:01:01.110
```

(1 row(s) affected)

Especificar la función de sistema escalar como date

En este ejemplo se especifica SYSDATETIME para date. El valor devuelto exacto depende del día y la hora de ejecución de la instrucción:

```
SELECT DATEADD(month, 1, SYSDATETIME());
```

El conjunto de resultados es el siguiente:

```
-----  
2013-02-06 14:29:59.6727944
```

(1 row(s) affected)

Especificar subconsultas y funciones escalares como number y date

En este ejemplo se usan subconsultas escalares, MAX(ModifiedDate), como argumentos para number y date. (SELECT TOP 1 BusinessEntityID FROM Person.Person) actúa como argumento artificial para el parámetro "number", para mostrar cómo se selecciona un argumento number de una lista de valores.

```
SELECT DATEADD(month,(SELECT TOP 1 BusinessEntityID FROM Person.Person),  
    (SELECT MAX(ModifiedDate) FROM Person.Person));
```

Especificar expresiones numéricas y funciones del sistema escalares como number y date

En este ejemplo se usa una expresión numérica $-(10/2)$, operadores unarios (-), un operador aritmético (/) y funciones del sistema escalares (SYSDATETIME) como argumentos para number y date.

```
SELECT DATEADD(month,-(10/2), SYSDATETIME());
```

Especificar las funciones de clasificación como number

En este ejemplo se usa una función de categoría como argumento para number.

```
SELECT p.FirstName, p.LastName  
    ,DATEADD(day,ROW_NUMBER() OVER (ORDER BY  
        a.PostalCode),SYSDATETIME()) AS 'Row Number'  
FROM Sales.SalesPerson AS s  
    INNER JOIN Person.Person AS p  
        ON s.BusinessEntityID = p.BusinessEntityID  
    INNER JOIN Person.Address AS a  
        ON a.AddressID = p.BusinessEntityID  
WHERE TerritoryID IS NOT NULL
```

AND SalesYTD <> 0;

Especificar una función de ventana agregada como number

En este ejemplo se usa una función de ventana agregada como argumento para number.

```
SELECT SalesOrderID, ProductID, OrderQty
      ,DATEADD(day,SUM(OrderQty)
      OVER(PARTITION BY SalesOrderID),SYSDATETIME())) AS 'Total'
FROM Sales.SalesOrderDetail
WHERE SalesOrderID IN(43659,43664);
GO
```

DATEDIFF

Esta función devuelve el recuento (como un valor entero con firma) de los límites datepart que se han cruzado entre los valores startdate y enddate especificad

Sintaxis

DATEDIFF (datepart , startdate , enddate)

Argumentos

datepart

Las unidades en las que DATEDIFF informa la diferencia entre startdate y enddate. Entre las unidades de datepart usadas comúnmente se incluyen month o second.

No se puede especificar el valor datepart en una variable, ni como una cadena entrecomillada, como 'month'.

En esta tabla se enumeran todos los valores válidos de *datepart*. DATEDIFF acepta el nombre completo de *datepart* o cualquier abreviatura indicada del nombre completo.

nombre de <i>datepart</i>	abreviatura de <i>datepart</i>
year	Y, YY, YYYY
quarter	qq, q
month	mm, m
dayofyear	dy
day	dd, d
week	wk, ww
hour	hh
minute	mi, n
second	ss, s
millisecond	ms
microsecond	mcs
nanosecond	ns

Nota

Todos los nombres y las abreviaturas específicas de *datepart* para ese nombre de *datepart* devolverán el mismo valor.

startdate

Una expresión que se puede resolver en uno de los valores siguientes:

date

datetime

datetimeoffset

datetime2

smalldatetime

time

Para evitar ambigüedades, use años de cuatro dígitos. Vea Establecer la opción de configuración del servidor Fecha límite de año de dos dígitos para obtener información sobre los valores de año de dos dígitos.

enddate

Vea startdate.

Tipo de valor devuelto

int

Valor devuelto

La diferencia int entre startdate y enddate, expresada en el límite establecido por datepart.

Por ejemplo, `SELECT DATEDIFF(day, '2036-03-01', '2036-02-28');` devuelve -2, que indica que 2036 debe ser un año bisiesto. Este caso significa que si comenzamos en startdate "2036-03-01" y, luego, contamos -2 días, alcanzamos enddate "2036-02-28".

Para un valor devuelto fuera del intervalo de int (de -2.147.483.648 a +2.147.483.647) DATEDIFF devuelve un error. En millisecond, la diferencia máxima entre startdate y enddate es de 24 días, 20 horas, 31 minutos y 23.647 segundos.

Para second, la diferencia máxima es de 68 años, 19 días, 3 horas, 14 minutos y 7 segundos.

Si startdate y enddate solo tienen asignado un valor de hora y datepart no es un valor datepart de hora, DATEDIFF devuelve 0.

DATEDIFF no usa el componente de ajuste de zona horaria de startdate o enddate para calcular el valor devuelto.

Como smalldatetime solo es preciso hasta los minutos, los segundos y milisegundos siempre se establecen en 0 en el valor devuelto cuando startdate o enddate tienen un valor smalldatetime.

Si solo se asigna un valor de hora a una variable de tipo de datos de fecha, DATEDIFF establece el valor de la parte de la fecha que falta en el valor predeterminado: 1900-01-01. Si solo se asigna un valor de fecha a una variable de tipo de datos de fecha u hora, DATEDIFF establece el valor de la parte de la hora que falta en el valor predeterminado: 00:00:00. Si startdate o enddate solo tienen una parte de hora y el otro solo una parte de fecha, DATEDIFF establece las partes de hora y fecha que faltan en los valores predeterminados.

Si startdate y enddate tienen tipos de datos de fecha diferentes y uno tiene más partes de hora o precisión de fracciones de segundo que el otro, DATEDIFF establece las partes que faltan del otro en 0.

Límites de datepart

Las instrucciones siguientes tienen los mismos valores startdate y enddate. Esas fechas son adyacentes y tienen una diferencia horaria de cien nanosegundos

(0,0000001 segundos). La diferencia entre startdate y enddate en cada instrucción cruza un límite de calendario u hora de su datepart. Cada instrucción devuelve 1.

```
SELECT DATEDIFF(year, '2005-12-31 23:59:59.9999999', '2006-01-01
00:00:00.0000000');
SELECT DATEDIFF(quarter, '2005-12-31 23:59:59.9999999', '2006-01-01
00:00:00.0000000');
SELECT DATEDIFF(month, '2005-12-31 23:59:59.9999999', '2006-01-01
00:00:00.0000000');
SELECT DATEDIFF(dayofyear, '2005-12-31 23:59:59.9999999', '2006-01-01
00:00:00.0000000');
SELECT DATEDIFF(day, '2005-12-31 23:59:59.9999999', '2006-01-01
00:00:00.0000000');
SELECT DATEDIFF(week, '2005-12-31 23:59:59.9999999', '2006-01-01
00:00:00.0000000');
SELECT DATEDIFF(hour, '2005-12-31 23:59:59.9999999', '2006-01-01
00:00:00.0000000');
SELECT DATEDIFF(minute, '2005-12-31 23:59:59.9999999', '2006-01-01
00:00:00.0000000');
SELECT DATEDIFF(second, '2005-12-31 23:59:59.9999999', '2006-01-01
00:00:00.0000000');
SELECT DATEDIFF(millisecond, '2005-12-31 23:59:59.9999999', '2006-01-01
00:00:00.0000000');
SELECT DATEDIFF(microsecond, '2005-12-31 23:59:59.9999999', '2006-01-01
00:00:00.0000000');
```

Si startdate y enddate tienen valores de año diferentes, pero tienen los mismos valores de semana del calendario, DATEDIFF devolverá 0 para datepartweek.

Observaciones

Use DATEDIFF en las cláusulas SELECT <list>, WHERE, HAVING, GROUP BY y ORDER BY.

DATEDIFF convierte implícitamente los literales de cadena como un tipo datetime2. Esto significa que DATEDIFF no admite el formato año-día-mes cuando la fecha se pasa como una cadena. La cadena se debe convertir explícitamente a un tipo datetime o smalldatetime para poder usar el formato año-día-mes.

La especificación de SET DATEFIRST no tiene ningún efecto sobre DATEDIFF. DATEDIFF siempre usa el domingo como el primer día de la semana para garantizar que la función actúa de forma determinista.

DATEDIFF se puede desbordar con una precisión de minute o superior si la diferencia entre enddate y startdate devuelve un valor que está fuera del intervalo para int.

Ejemplos

En estos ejemplos se usan otros tipos de expresiones como argumentos para los parámetros startdate y enddate.

A. Especificar las columnas para startdate y enddate

En este ejemplo se calcula el número de límites de día que se cruzan entre las fechas en dos columnas de una tabla.

```
CREATE TABLE dbo.Duration  
(startDate datetime2, endDate datetime2);
```

```
INSERT INTO dbo.Duration(startDate, endDate)
VALUES ('2007-05-06 12:10:09', '2007-05-07 12:10:09');
```

```
SELECT DATEDIFF(day, startDate, endDate) AS 'Duration'
FROM dbo.Duration;
-- Returns: 1
```

B. Especificar las variables definidas por el usuario para startdate y enddate

En este ejemplo, las variables definidas por el usuario actúan como argumentos para startdate y enddate.

```
DECLARE @startdate DATETIME2 = '2007-05-05 12:10:09.3312722';
DECLARE @enddate DATETIME2 = '2007-05-04 12:10:09.3312722';
SELECT DATEDIFF(day, @startdate, @enddate);
```

C. Especificar las funciones de sistema escalares para startdate y enddate

En este ejemplo se usan las funciones de sistema escalares como argumentos para startdate y enddate.

```
SELECT DATEDIFF(second, GETDATE(), SYSDATETIME());
```

D. Especificar las funciones escalares y de subconsulta para startdate y enddate

En este ejemplo se usan las funciones escalares y de subconsulta como argumentos para startdate y enddate.

```
USE AdventureWorks2022;
GO
SELECT DATEDIFF(day,
```

```
(SELECT MIN(OrderDate) FROM Sales.SalesOrderHeader),  
(SELECT MAX(OrderDate) FROM Sales.SalesOrderHeader));
```

E. Especificar las constantes para startdate y enddate

En este ejemplo se usan constantes de caracteres como argumentos para startdate y enddate.

```
SELECT DATEDIFF(day,  
    '2007-05-07 09:53:01.0376635',  
    '2007-05-08 09:53:01.0376635');
```

F. Especificar expresiones numéricas y funciones de sistema escalares para enddate

En este ejemplo se usa una expresión numérica, (GETDATE() + 1), y las funciones de sistema escalares GETDATE y SYSDATETIME, como argumentos para enddate.

```
USE AdventureWorks2022;  
GO  
SELECT DATEDIFF(day, '2007-05-07 09:53:01.0376635', GETDATE() + 1)  
    AS NumberOfDays  
    FROM Sales.SalesOrderHeader;  
GO  
USE AdventureWorks2022;  
GO  
SELECT  
    DATEDIFF(  
        day,  
        '2007-05-07 09:53:01.0376635',
```

```
DATEADD(day, 1, SYSDATETIME())
) AS NumberOfDays
FROM Sales.SalesOrderHeader;
GO
```

G. Especificar las funciones de clasificación para startdate

En este ejemplo se usa una función de categoría como argumento para startdate.

```
USE AdventureWorks2022;
GO
SELECT p.FirstName, p.LastName
      ,DATEDIFF(day, ROW_NUMBER() OVER (ORDER BY
      a.PostalCode), SYSDATETIME()) AS 'Row Number'
FROM Sales.SalesPerson s
      INNER JOIN Person.Person p
      ON s.BusinessEntityID = p.BusinessEntityID
      INNER JOIN Person.Address a
      ON a.AddressID = p.BusinessEntityID
WHERE TerritoryID IS NOT NULL
      AND SalesYTD <> 0;
```

H. Especificar una función de ventana agregada para startdate

En este ejemplo se usa una función de ventana agregada como argumento para startdate

```
USE AdventureWorks2022;
```



```

GO
SELECT soh.SalesOrderID, sod.ProductID, sod.OrderQty, soh.OrderDate,
       DATEDIFF(day, MIN(soh.OrderDate)
               OVER(PARTITION BY soh.SalesOrderID), SYSDATETIME()) AS 'Total'
FROM Sales.SalesOrderDetail sod
     INNER JOIN Sales.SalesOrderHeader soh
           ON sod.SalesOrderID = soh.SalesOrderID
WHERE soh.SalesOrderID IN(43659, 58918);
GO

```

- I. Búsqueda de la diferencia entre startdate y enddate como cadenas de partes de fecha

-- DOES NOT ACCOUNT FOR LEAP YEARS

```
DECLARE @date1 DATETIME, @date2 DATETIME, @result VARCHAR(100);
```

```
DECLARE @years INT, @months INT, @days INT,
```

```
        @hours INT, @minutes INT, @seconds INT, @milliseconds INT;
```

```
SET @date1 = '1900-01-01 00:00:00.000'
```

```
SET @date2 = '2018-12-12 07:08:01.123'
```

```
SELECT @years = DATEDIFF(yy, @date1, @date2)
```

```
IF DATEADD(yy, -@years, @date2) < @date1
```

```
SELECT @years = @years-1
```

```
SET @date2 = DATEADD(yy, -@years, @date2)
```

```
SELECT @months = DATEDIFF(mm, @date1, @date2)
```

```
IF DATEADD(mm, -@months, @date2) < @date1
```

```
SELECT @months=@months-1
SET @date2= DATEADD(mm, -@months, @date2)
```

```
SELECT @days=DATEDIFF(dd, @date1, @date2)
IF DATEADD(dd, -@days, @date2) < @date1
SELECT @days=@days-1
SET @date2= DATEADD(dd, -@days, @date2)
```

```
SELECT @hours=DATEDIFF(hh, @date1, @date2)
IF DATEADD(hh, -@hours, @date2) < @date1
SELECT @hours=@hours-1
SET @date2= DATEADD(hh, -@hours, @date2)
```

```
SELECT @minutes=DATEDIFF(mi, @date1, @date2)
IF DATEADD(mi, -@minutes, @date2) < @date1
SELECT @minutes=@minutes-1
SET @date2= DATEADD(mi, -@minutes, @date2)
```

```
SELECT @seconds=DATEDIFF(s, @date1, @date2)
IF DATEADD(s, -@seconds, @date2) < @date1
SELECT @seconds=@seconds-1
SET @date2= DATEADD(s, -@seconds, @date2)
```

```
SELECT @milliseconds=DATEDIFF(ms, @date1, @date2)
```

```
SELECT @result= ISNULL(CAST(NULLIF(@years,0) AS VARCHAR(10)) + ' years,',")
+ ISNULL(' ' + CAST(NULLIF(@months,0) AS VARCHAR(10)) + ' months,',")
```

```

+ ISNULL(' ' + CAST(NULLIF(@days,0) AS VARCHAR(10)) + ' days,','")
+ ISNULL(' ' + CAST(NULLIF(@hours,0) AS VARCHAR(10)) + ' hours,','")
+ ISNULL(' ' + CAST(@minutes AS VARCHAR(10)) + ' minutes and,','")
+ ISNULL(' ' + CAST(@seconds AS VARCHAR(10))
+ CASE
    WHEN @milliseconds > 0
        THEN '.' + CAST(@milliseconds AS VARCHAR(10))
    ELSE ''
END
+ ' seconds','")

```

SELECT @result

El conjunto de resultados es el siguiente:

118 years, 11 months, 11 days, 7 hours, 8 minutes and 1.123 seconds

En estos ejemplos se usan otros tipos de expresiones como argumentos para los parámetros startdate y enddate.

J. Especificar las columnas para startdate y enddate

En este ejemplo se calcula el número de límites de día que se cruzan entre las fechas en dos columnas de una tabla.

```

CREATE TABLE dbo.Duration
(
    startDate datetime2, endDate datetime2);

```

```
INSERT INTO dbo.Duration (startDate, endDate)
VALUES ('2007-05-06 12:10:09', '2007-05-07 12:10:09');
```

```
SELECT TOP(1) DATEDIFF(day, startDate, endDate) AS Duration
FROM dbo.Duration;
```

-- Returns: 1

K. Especificar las funciones escalares y de subconsulta para startdate y enddate

En este ejemplo se usan las funciones escalares y de subconsulta como argumentos para startdate y enddate.

-- Uses AdventureWorks

```
SELECT TOP(1) DATEDIFF(day, (SELECT MIN(HireDate) FROM dbo.DimEmployee),
    (SELECT MAX(HireDate) FROM dbo.DimEmployee))
FROM dbo.DimEmployee;
```

L. Especificar las constantes para startdate y enddate

En este ejemplo se usan constantes de caracteres como argumentos para startdate y enddate.

-- Uses AdventureWorks

```
SELECT TOP(1) DATEDIFF(day,
    '2007-05-07 09:53:01.0376635',
    '2007-05-08 09:53:01.0376635') FROM DimCustomer;
```

M. Especificar las funciones de clasificación para startdate

En este ejemplo se usa una función de categoría como argumento para startdate.

-- Uses AdventureWorks

```
SELECT FirstName, LastName,  
       DATEDIFF(day, ROW_NUMBER() OVER (ORDER BY  
       DepartmentName), SYSDATETIME()) AS RowNumber  
FROM dbo.DimEmployee;
```

Hora Especificar una función de ventana agregada para startdate

En este ejemplo se usa una función de ventana agregada como argumento para startdate.

-- Uses AdventureWorks

```
SELECT FirstName, LastName, DepartmentName,  
       DATEDIFF(year, MAX(HireDate)  
       OVER (PARTITION BY DepartmentName), SYSDATETIME()) AS SomeValue  
FROM dbo.DimEmployee
```

DATEPART

Esta función devuelve un entero que representa el parámetro datepart especificado del parámetro date especificado.

Sintaxis

DATEPART (datepart , date)

Argumentos

datepart

La parte específica del argumento date para el que DATEPART va a devolver un valor integer. En esta tabla se enumeran todos los argumentos válidos de datepart.

Nota

DATEPART no acepta los equivalentes de variables definidas por el usuario para los argumentos datepart.

<i>datepart</i>	Abreviaturas
year	yy, yyyy
quarter	qq, q
month	mm, m
dayofyear	dy, y
day	dd, d
week	wk, ww
weekday	dw
hour	hh
minute	mi, n
second	ss, s
millisecond	ms
microsecond	mcs
nanosecond	ns
tzoffset	tz
iso_week	isowk, isoww

date

Una expresión que se resuelve en uno de los tipos de datos siguientes:

date

datetime

datetimeoffset

datetime2

smalldatetime

time

Para date, DATEPART aceptará una expresión de columna, una expresión, un literal de cadena o una variable definida por el usuario. Para evitar problemas de ambigüedad, use años de cuatro dígitos. Vea Establecer la opción de configuración del servidor Fecha límite de año de dos dígitos para obtener información sobre los años de dos dígitos.

Tipo de valor devuelto

int

Valor devuelto

Cada datepart y sus abreviaturas devuelven el mismo valor.

El valor devuelto depende del entorno del idioma definido mediante SET LANGUAGE y la opción de configuración de servidor Configurar el idioma predeterminado del inicio de sesión. El valor devuelto depende de SET DATEFORMAT si date es un literal de cadena de ciertos formatos. SET DATEFORMAT no cambia el valor devuelto cuando la fecha es una expresión de columna de un tipo de datos de hora o fecha.

En esta tabla se enumeran todos los argumentos datepart, con los correspondientes valores devueltos para la instrucción SELECT DATEPART(datepart,'2007-10-30

12:15:32.1234567 +05:10'). El argumento date tiene un tipo de datos datetimeoffset(7) . Las dos últimas posiciones del valor devuelto datepartnanosecond siempre son 00 y este valor tiene una escala de 9:

0,123456700

<i>datepart</i>	Valor devuelto
year, yyyy, yy	2007
quarter, qq, q	4
month, mm, m	10
dayofyear, dy, y	303
day, dd, d	30
week, wk, ww	44
weekday, dw	3
hour, hh	12
minute, n	15
second, ss, s	32
millisecond, ms	123
microsecond, mcs	123456
nanosecond, ns	123456700
tzoffset, tz	310
iso_week, isowk, isoww	44

Argumentos de la parte de fecha semana y día de la semana

Si datepart es week (wk, ww) o weekday (dw), el valor devuelto de DATEPART depende del valor establecido mediante SET DATEFIRST.

El 1 de enero de todos los años es el número de inicio para datepartweek. Por ejemplo:

DATEPART (wk, "Jan 1, xxxx") = 1

donde xxxx es cualquier año.

En esta tabla se muestran los valores devueltos para datepartweek y weekday para "2007-04-21" para cada argumento de SET DATEFIRST. El 1 de enero de 2007 es lunes. El 21 de abril de 2007 es un sábado. Para Inglés de EE.UU.,

SET DATEFIRST 7 -- (Sunday)

sirve como valor predeterminado. Después de establecer DATEFIRST, use esta instrucción SQL sugerida para los valores de tabla de datepart:

SELECT DATEPART(week, '2007-04-21 '), DATEPART(weekday, '2007-04-21 ')

SET	DATEFIRST	week	weekday
Argumento		devuelto	devuelto
1		16	6
2		17	5
3		17	4
4		17	3
5		17	2

6	17	1
7	16	7

Argumentos de datepart year, month y day

Los valores devueltos para DATEPART (year, date), DATEPART (month, date) y DATEPART (day, date) son los mismos que los que devuelven las funciones YEAR, MONTH y DAY, respectivamente.

iso_week datepart

ISO 8601 incluye el sistema ISO de fecha-semana, un sistema de numeración para las semanas. Cada semana se asocia al año en el que cae el jueves. Por ejemplo, la semana 1 de 2004 (2004W01) abarcaba del lunes 29 de diciembre de 2003 al domingo 4 de enero de 2004. En los países o regiones Europeos normalmente se usa este estilo de numeración. En los países o regiones que no son europeos normalmente no se usa.

Nota: El número más alto de la semana en un año puede ser 52 o 53.

Es posible que los sistemas de numeración que se usan en otros países o regiones no se ajusten a las normas ISO. En esta tabla se muestran seis posibilidades:

Primer día de la semana	La primera semana del año contiene	Semanas asignadas dos veces	Usado por/en
Domingo	1 de enero, El primer sábado, 1-7 días del año	Sí	Estados Unidos

Lunes	1 de enero, El primer domingo, 1-7 días del año	Sí	La mayoría de los países Europeos y Reino Unido
Lunes	4 de enero, El primer jueves, 4-7 días del año	No	ISO 8601, Noruega y Suecia
Lunes	7 de enero, El primer lunes, Siete días del año	No	
Miércoles	1 de enero, El primer martes, 1-7 días del año	Sí	
Sábado	1 de enero, El primer viernes, 1-7 días del año	Sí	

tzoffset

DATEPART devuelve el valor tzoffset (tz) como el número de minutos (con signo).

Esta instrucción devuelve un desplazamiento de zona horaria de 310 minutos:

```
SELECT DATEPART (tzoffset, '2007-05-10 00:00:01.1234567 +05:10');
```

DATEPART representa el valor tzoffset de esta forma:

- Para datetimeoffset y datetime2, tzoffset devuelve el desplazamiento de tiempo en minutos, donde el desplazamiento de datetime2 siempre es 0 minutos.
- Para los tipos de datos que se pueden convertir implícitamente en datetimeoffset o datetime2, DATEPART devuelve el desplazamiento de tiempo en minutos. Excepción: otros tipos de datos de fecha y hora.
- El resto de tipos de parámetros producirán un error.

Argumento date smalldatetime

Para un valor datesmalldatetime, DATEPART devuelve los segundos como 00.

Valor predeterminado devuelto por una parte de fecha que no se encuentra en un argumento date

Si el tipo de datos del argumento date no contiene el parámetro datepart especificado, DATEPART devolverá el valor predeterminado para ese parámetro datepart solo cuando se especifique un valor literal para date.

Por ejemplo, el valor predeterminado de año-mes-día de cualquier tipo de datos date es 1900-01-01. Esta instrucción tiene argumentos de la parte de fecha para datepart, un argumento de hora para date y devuelve 1900, 1, 1, 1, 2.

```
SELECT DATEPART(year, '12:10:30.123')
      ,DATEPART(month, '12:10:30.123')
      ,DATEPART(day, '12:10:30.123')
      ,DATEPART(dayofyear, '12:10:30.123')
      ,DATEPART(weekday, '12:10:30.123');
```

Si date se especifica como variable o columna de tabla, y el tipo de datos de esa variable o columna no tiene especificado datepart, DATEPART devuelve el error 9810. En este ejemplo, la variable @t tiene un tipo de datos time. Se produce un error en el ejemplo porque el año de la parte de fecha no es válido para el tipo de datos time:

```
DECLARE @t time = '12:10:30.123';  
SELECT DATEPART(year, @t);
```

Si date se especifica como variable o columna de tabla, y el tipo de datos de esa variable o columna no tiene especificado datepart, DATEPART devuelve el error 9810. En este ejemplo, la variable @t tiene un tipo de datos time. Se produce un error en el ejemplo porque el año de la parte de fecha no es válido para el tipo de datos time:

```
DECLARE @t time = '12:10:30.123';  
SELECT DATEPART(year, @t);
```

Fracciones de segundo

En estas instrucciones se muestra que DATEPART devuelve las fracciones de segundo:

```
SELECT DATEPART(millisecond, '00:00:01.1234567'); -- Returns 123  
SELECT DATEPART(microsecond, '00:00:01.1234567'); -- Returns 123456  
SELECT DATEPART(nanosecond, '00:00:01.1234567'); -- Returns 123456700
```

Observaciones

DATEPART se puede usar en las cláusulas SELECT list, WHERE, HAVING, GROUP BY y ORDER BY.

DATEPART convierte de forma implícita los literales de cadena como un tipo datetime2 en SQL Server 2008 (10.0.x) y versiones posteriores. Esto significa que DATENAME no admite el formato año-día-mes cuando se pasa la fecha como cadena. La cadena se debe convertir explícitamente a un tipo datetime o smalldatetime para poder usar el formato año-día-mes.

Ejemplos

En este ejemplo se devuelve el año base. El año base ayuda con los cálculos de fecha. En el ejemplo, un número especifica la fecha. Observe que SQL Server interpreta 0 como el 1 de enero de 1900.

```
SELECT DATEPART(year, 0), DATEPART(month, 0), DATEPART(day, 0);
```

```
-- Returns: 1900    1    1
```

En este ejemplo se devuelve el día de la fecha 12/20/1974.

```
-- Uses AdventureWorks
```

```
SELECT TOP(1) DATEPART (day,'12/20/1974') FROM dbo.DimCustomer;
```

```
-- Returns: 20
```

En este ejemplo se devuelve el año de la fecha 12/20/1974.

-- Uses AdventureWorks

```
SELECT TOP(1) DATEPART (year,'12/20/1974') FROM dbo.DimCustomer;
```

-- Returns: 1974

ISDATE

Devuelve 1 si expression es un valor válido de datetime; en caso contrario, devuelve 0.

ISDATE devuelve 0 si expression es un valor datetime2.

Sintaxis

ISDATE (expression)

Argumentos

expression

Es una cadena de caracteres o una expresión que se puede convertir en una cadena de caracteres. La expresión debe tener menos de 4.000 caracteres. No se permiten tipos de datos de fecha y hora, excepto datetime y smalldatetime, como argumento para ISDATE.

Tipo de valor devuelto

int

Observaciones

ISDATE solo es determinista si se usa con la función CONVERT, se especifica el parámetro de estilo CONVERT y el estilo no es igual a 0, 100, 9 ni 109.

El valor devuelto de ISDATE depende de los valores establecidos por SET DATEFORMAT, SET LANGUAGE y la opción de configuración del servidor Idioma predeterminado.

Formatos de expresión ISDATE

Para obtener ejemplos de formatos válidos para los que ISDATE devolverá 1, vea la sección "Formatos de literales de cadena compatibles para datetime" en los temas datetime y smalldatetime. Para obtener más ejemplo, vea también la columna Entrada/salida de la sección "Argumentos" de CAST y CONVERT.

En la tabla siguiente se resumen los formatos de expresión de entrada que no son válidos y devuelven 0 o un error.

Expresión ISDATE	Valor devuelto de ISDATE
NULL	0
Valores de tipos de datos incluidos en la lista Tipos de datos en cualquier categoría de tipo de datos distinta de las cadenas de caracteres, cadenas de caracteres Unicode o fecha y hora.	0
Valores de los tipos de datos text , ntexto image .	0
Cualquier valor que tiene una escala de precisión por segundos mayor que 3 (.0000 a.00000000... n) ISDATE devuelve 0 si <i>expression</i> es un valor datetime2 , pero devolverá 1 si <i>expression</i> es un valor datetime válido.	0

Cualquier valor que mezcla una fecha válida con un valor no válido, por ejemplo 1995-10-1a.	0
---	---

Ejemplos

A. Utilizar ISDATE para probar si una expresión de fecha y hora es válida

En este ejemplo se muestra cómo usar ISDATE para probar si una cadena de caracteres es un tipo datetime válido.

```
IF ISDATE('2009-05-12 10:19:41.177') = 1
    PRINT 'VALID'
ELSE
    PRINT 'INVALID';
```

B. Mostrar los efectos de los parámetros SET DATEFORMAT y SET LANGUAGE en los valores devueltos

Las instrucciones siguientes muestran los valores que se devuelven al establecer SET DATEFORMAT y SET LANGUAGE.

```
/* Use these sessions settings. */
SET LANGUAGE us_english;
SET DATEFORMAT mdy;
/* Expression in mdy dateformat */
SELECT ISDATE('04/15/2008'); --Returns 1.
/* Expression in mdy dateformat */
SELECT ISDATE('04-15-2008'); --Returns 1.
/* Expression in mdy dateformat */
SELECT ISDATE('04.15.2008'); --Returns 1.
```

```
/* Expression in myd dateformat */  
SELECT ISDATE('04/2008/15'); --Returns 1.
```

```
SET DATEFORMAT mdy;  
SELECT ISDATE('15/04/2008'); --Returns 0.  
SET DATEFORMAT mdy;  
SELECT ISDATE('15/2008/04'); --Returns 0.  
SET DATEFORMAT mdy;  
SELECT ISDATE('2008/15/04'); --Returns 0.  
SET DATEFORMAT mdy;  
SELECT ISDATE('2008/04/15'); --Returns 1.
```

```
SET DATEFORMAT dmy;  
SELECT ISDATE('15/04/2008'); --Returns 1.  
SET DATEFORMAT dym;  
SELECT ISDATE('15/2008/04'); --Returns 1.  
SET DATEFORMAT ydm;  
SELECT ISDATE('2008/15/04'); --Returns 1.  
SET DATEFORMAT ymd;  
SELECT ISDATE('2008/04/15'); --Returns 1.
```

```
SET LANGUAGE English;  
SELECT ISDATE('15/04/2008'); --Returns 0.  
SET LANGUAGE Hungarian;  
SELECT ISDATE('15/2008/04'); --Returns 0.  
SET LANGUAGE Swedish;  
SELECT ISDATE('2008/15/04'); --Returns 0.
```

```
SET LANGUAGE Italian;  
SELECT ISDATE('2008/04/15'); --Returns 1.
```

```
/* Return to these sessions settings. */  
SET LANGUAGE us_english;  
SET DATEFORMAT mdy;
```

C. Utilizar ISDATE para probar si una expresión de fecha y hora es válida

En este ejemplo se muestra cómo usar ISDATE para probar si una cadena de caracteres es un tipo datetime válido.

```
IF ISDATE('2009-05-12 10:19:41.177') = 1  
    SELECT 'VALID';  
ELSE  
    SELECT 'INVALID';
```

Otras Funciones

CAST y CONVERT

Estas funciones convierten una expresión de un tipo de datos a otro.

Sintaxis

Sintaxis CAST:

```
CAST ( expression AS data_type [ ( length ) ] )
```

Sintaxis CONVERT:

```
CONVERT ( data_type [ ( length ) ] , expression [ , style ] )
```

Argumentos

expression

Cualquier expression válida.

data_type

El tipo de datos de destino. Esto engloba xml, bigint y sql_variant. No se pueden utilizar tipos de datos de alias.

length

Entero opcional que especifica la longitud del tipo de datos de destino para los tipos de datos que permiten una longitud especificada por el usuario. El valor predeterminado es 30.

style

Una expresión de tipo entero que especifica cómo traducirá la función CONVERT la expresión. Para un valor de estilo NULL, se devuelve NULL. data_type determina el intervalo.

Tipos de valores devueltos

Devuelve expression, traducido a data_type.

Estilos de fecha y hora

Para una expression que tenga el tipo de datos de fecha u hora, style puede tener uno de los valores que se muestran en la siguiente tabla. Otros valores se procesan como 0. A partir de SQL Server 2012 (11.x), los únicos estilos que se admiten al convertir de tipos de fecha y hora a datetimeoffset son 0 o 1. Todos los demás estilos de conversión devuelven el error 9809.

Nota

SQL Server admite el formato de fecha, en estilo árabe, con el algoritmo kuwaití.

Sin el siglo (yy) ¹	Con el siglo (aaaa)	Estándar	Entrada/salida (³)
-	0 o 100 ^{1,2}	Valor predeterminado para datetime y smalldatetime	mon dd yyyy hh:miAM (o PM)
1	101	EE. UU.	1 = mm/dd/yy 101 = mm/dd/yyyy
2	102	ANSI	2 = yy.mm.dd 102 = yyyy.mm.dd
3	103	Británico/Francés	3 = dd/mm/yy 103 = dd/mm/yyyy
4	104	Alemán	4 = dd.mm.yy 104 = dd.mm/yyyy
5	105	Italiano	5 = dd-mm-yy 105 = dd-mm-yyyy
6	106 ¹	-	6 = dd mon yy 106 = dd mon yyyy
7	107 ¹	-	7 = Mon dd, yy 107 = Mon dd, yyyy
8 o 24	108	-	hh:mi:ss
-	9 o 109 ^{1,2}	Valor predeterminado + milisegundos	mon dd yyyy hh:mi:ss:mmmAM (o PM)
10	110	EE. UU.	10 = mm-dd-aa 110 = mm-dd-yyyy
11	111	JAPÓN	11 = aa/mm/dd 111 = yyyy/mm/dd

12	112	ISO	12 = aammdd 112 = yyyyymmdd
-	13 or 113 ^{1,2}	Europeo predeterminado + milisegundos	dd mon yyyy hh:mi:ss:mmm (24 horas)
14	114	-	hh:mi:ss:mmm (24 horas)
-	20 o 120 ²	ODBC canónico	yyyy-mm-dd hh:mi:ss (24 horas)
-	21 o 25 o 121 ²	ODBC canónico (con milisegundos), valor predeterminado para time , date , datetime y datetimeoffset	yyyy-mm-dd hh:mi:ss:mmm (24 horas)
22	-	EE. UU.	mm/dd/yy hh:mi:ss AM (o PM)
-	23	ISO8601	yyyy-mm-dd
-	126 ⁴	ISO8601	yyyy-mm-ddThh:mi:ss:mmm (sin espacios) ⁶
-	127 ^{8, 9}	ISO8601 con zona horaria Z	yyyy-MM-ddThh:mm:ss.fffZ (sin espacios) ⁶
-	130 ^{1,2}	Hijri ⁵	dd mon yyyy hh:mi:ss:mmmAM ⁷
-	131 ²	Hijri ⁵	dd/mm/yyyy hh:mi:ss:mmmAM

1 Estos valores de estilo devuelven resultados no deterministas. Incluye todos los estilos (yy) (sin el siglo) y un subconjunto de estilos (yyyy) (con el siglo).

2 Los valores predeterminados (0 o 100, 9 o 109, 13 o 113, 20 o 120, 23 y 21 o 25 o 121) siempre devuelven el siglo (yyyy).

Importante

De forma predeterminada, SQL Server interpreta los años de dos dígitos según el año límite 2049. Esto significa que SQL Server interpreta el año 49, de dos dígitos,

como 2049, y el año 50, de dos dígitos, como 1950. Muchas aplicaciones cliente, como las que se basan en objetos de Automation, usan el año límite de 2030. SQL Server proporciona la opción de configuración de año límite de dos dígitos para cambiar el año límite usado por SQL Server. Esto permite tratar las fechas de manera coherente. Se recomienda especificar años de cuatro dígitos.

3 Entrada cuando se convierte en datetime; salida cuando se convierte en datos de caracteres.

4 Diseñado para usarse con XML. Para convertir datos datetime o smalldatetime en datos de caracteres, consulte la tabla anterior para ver el formato de salida.

5 Hijri es un sistema del calendario con varias variaciones. SQL Server utiliza el algoritmo kuwaití.

6 En el caso de un valor 0 en milisegundos (mmm), el valor de fracción decimal en milisegundos no se mostrará. Por ejemplo, el valor 2022-11-07T18:26:20.000 se muestra como 2022-11-07T18:26:20.

7 En este estilo, mon es una representación Unicode Hijri multitoken del nombre completo del mes. Este valor no se representa correctamente en una instalación estadounidense predeterminada de SSMS.

8 Solo se admite en la conversión de datos de caracteres a datetime o smalldatetime. Al convertir datos de caracteres que representan componentes de solo fecha o solo hora al tipo de datos datetime o smalldatetime, el componente de hora no especificado se establece en 00:00:00.000 y el componente de fecha no especificado se establece en 1900-01-01.

9 Use el indicador opcional de zona horaria Z para facilitar la asignación de valores XML de tipo datetime que contienen información de zona horaria a valores de tipo SQL Server datetime que no tienen zona horaria. Z indica la zona horaria en UTC-0. El desplazamiento HH:MM, en sentido + o -, indica otras zonas horarias. Por ejemplo: 2022-12-12T23:45:12-08:00.

Al convertir datos smalldatetime en datos de caracteres, los estilos que incluyen segundos o milisegundos muestran ceros en dichas posiciones. Al convertir valores datetime o smalldatetime, use una longitud adecuada de valor de datos char o varchar para truncar las partes de la fecha que no quiera.

Al convertir datos de caracteres en datos datetimeoffset con un estilo que incluye una hora, se anexa un desplazamiento de zona horaria al resultado.

Estilos float y real

En el caso de una expressionfloat o real, style puede tener uno de los valores que se muestran en la siguiente tabla. Otros valores se procesan como 0.

Value	Output
0 (valor predeterminado)	Un máximo de 6 dígitos. Utilícelo en notación científica cuando proceda.
1	Siempre 8 dígitos. Utilícelo siempre en notación científica.
2	Siempre 16 dígitos. Utilícelo siempre en notación científica.
3	Siempre 17 dígitos. Se usa para la conversión sin pérdida de información. Con este estilo, se garantiza que cada valor de float o real distinto se va a convertir en una cadena de caracteres distinta.

	Se aplica a: SQL Server 2016 (13.x) y versiones posteriores, y Azure SQL Database.
126, 128, 129	Se incluye por motivos de herencia. No use estos valores para una nueva implementación.

Estilos money y smallmoney

En el caso de una `expressionmoney` o `smallmoney`, `style` puede tener uno de los valores que se muestran en la siguiente tabla. Otros valores se procesan como 0.

Value	Output
0 (valor predeterminado)	Sin separadores de millar cada tres dígitos a la izquierda del separador decimal y dos dígitos a la derecha del separador decimal Ejemplo: 4235,98.
1	Separadores de millar cada tres dígitos a la izquierda del separador decimal y dos dígitos a la derecha del separador decimal Ejemplo: 3.510,92.
2	Sin separadores de millar cada tres dígitos a la izquierda del separador decimal y cuatro dígitos a la derecha del separador decimal Ejemplo: 4235,9819.
126	Equivalente al estilo 2 al convertir a char(<i>n</i>) o varchar(<i>n</i>)

Estilos xml

En el caso de una `expressionxml`, `style` puede tener uno de los valores que se muestran en la siguiente tabla. Otros valores se procesan como 0.

Value	Output
0 (valor predeterminado)	Usa el comportamiento de análisis predeterminado que descarta los espacios en blanco insignificantes y no permite un subconjunto DTD interno. Nota: Al convertir al tipo de datos xml, los espacios en blanco insignificantes de SQL Server se controlan de forma distinta que en XML 1.0. Para más información, vea Crear instancias de datos XML.
1	Conserva los espacios en blanco insignificantes. Esta configuración de estilo establece el control predeterminado de <code>xml:space</code> para emular el comportamiento de <code>xml:space="preserve"</code>.
2	Habilita el procesamiento limitado de subconjuntos DTD internos. Si está habilitado, el servidor puede usar la siguiente información proporcionada en un subconjunto DTD interno para realizar operaciones de análisis que no se validan. - Se aplican los valores predeterminados de los atributos - Las referencias a entidades internas se resuelven y se amplían - Se comprueba la corrección sintáctica del modelo de contenido DTD El analizador pasa por alto los subconjuntos DTD externos.

	Además, no evalúa la declaración XML para comprobar si el atributo <i>standalone</i> tiene el valor yes o no . En su lugar, analiza la instancia XML como documento independiente.
3	Conserva los espacios en blanco insignificantes y habilita el procesamiento limitado de los subconjuntos DTD internos.

Estilos binarios

En el caso de una expresión `binary(n)`, `char(n)`, `varbinary(n)` o `varchar(n)`, el valor `style` puede tener uno de los valores que se muestran en la siguiente tabla. Los valores de estilo que no figuran en la tabla devolverán un error.

Value	Output
0 (valor predeterminado)	Traduce caracteres ASCII a bytes binarios o bytes binarios a caracteres ASCII. Cada carácter o byte se convierte con una proporción 1:1. En el caso de un <i>data_type</i> binario, los caracteres 0x se agregan a la izquierda del resultado.
1, 2	Para un <i>data_type</i> binario, la expresión debe ser de caracteres. <i>expression</i> debe tener un número <i>par</i> de dígitos hexadecimales (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, a, b, c, d, e, f). Si <i>style</i> se establece en 1, los dos primeros caracteres de la expresión deben ser 0x. Si la expresión contiene un número impar de caracteres o si alguno de los caracteres no es válido, se producirá un error. Si la longitud de la expresión convertida supera la longitud de <i>data_type</i> , el resultado se truncará a la derecha. Los valores <i>data_type</i> de longitud fija que sean mayores que el resultado convertido tendrán ceros agregados a la derecha del resultado. Los <i>data_type</i> con el tipo carácter necesitan una expresión binaria. Cada carácter binario se convierte en dos caracteres hexadecimales. Imagine que la longitud de la expresión convertida supera la longitud de <i>data_type</i> . En ese caso, se trunca.

	Para un valor <i>data_type</i> de tipo de caracteres de tamaño fijo, si la longitud del resultado convertido es menor que la longitud de <i>data_type</i>, se agregan espacios a la derecha de la expresión convertida para mantener un número par de dígitos hexadecimales. Los caracteres 0x no se agregan a la izquierda del resultado convertido para <i>style</i> 2.
--	---

Conversiones implícitas

Las conversiones implícitas no requieren la especificación de la función CAST ni de la función CONVERT. Las conversiones explícitas requieren la especificación de la función CAST o de la función CONVERT. En la siguiente ilustración se muestran todas las conversiones de tipos de datos explícitas e implícitas permitidas para los tipos de datos proporcionados por el sistema de SQL Server. Algunas de ellas son bigint, sql_variant y xml. No existe una conversión implícita en la asignación del tipo de datos sql_variant, pero sí hay una conversión implícita en sql_variant.

Sugerencia

Este gráfico está disponible como archivo PNG para descargar en el Centro de descarga de Microsoft.

From \ To	binary	varbinary	char	varchar	nchar	nvarchar	datetime	smalldatetime	date	time	datetimeoffset	datetime2	decimal	numeric	float	real	bigint	int(INT4)	smallint(INT2)	tinyint(INT1)	money	smallmoney	bit	timestamp	uniqueidentifier	image	ntext	text	sql_variant	xml	CLR UDT	hierarchyid
binary		●	●	●	●	●	●	●	■	■	■	■	●	●	✗	✗	●	●	●	●	●	●	●	●	●	✗	✗	●	●	●	●	●
varbinary	●		●	●	●	●	●	●	■	■	■	■	●	●	✗	✗	●	●	●	●	●	●	●	●	●	✗	✗	●	●	●	●	●
char	■	■		●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	■	●	●	●	●	●	●	●	●	●
varchar	■	■	●		●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	■	●	●	●	●	●	●	●	●	●
nchar	■	■	●	●		●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	■	●	✗	●	●	●	●	●	●	●
nvarchar	■	■	●	●	●		●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	■	●	✗	●	●	●	●	●	●	●
datetime	■	■	●	●	●	●		●	●	●	●	■	■	■	■	■	■	■	■	■	■	■	■	■	✗	✗	✗	✗	●	✗	✗	✗
smalldatetime	■	■	●	●	●	●	●		●	●	●	■	■	■	■	■	■	■	■	■	■	■	■	■	✗	✗	✗	✗	●	✗	✗	✗
date	■	■	●	●	●	●	●	●		✗	●	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	●	✗	✗	✗
time	■	■	●	●	●	●	●	✗		●	●	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	●	✗	✗	✗
datetimeoffset	■	■	●	●	●	●	●	●	●		●	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	●	✗	✗	✗
datetime2	■	■	●	●	●	●	●	●	●	●		✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	●	✗	✗	✗
decimal	●	●	●	●	●	●	●	✗	✗	✗	✗	◆	◆	●	●	●	●	●	●	●	●	●	●	✗	✗	✗	✗	●	✗	✗	✗	✗
numeric	●	●	●	●	●	●	●	✗	✗	✗	✗	◆	◆	●	●	●	●	●	●	●	●	●	●	✗	✗	✗	✗	●	✗	✗	✗	✗
float	●	●	●	●	●	●	●	✗	✗	✗	✗	●	●		●	●	●	●	●	●	●	●	●	✗	✗	✗	✗	✗	●	✗	✗	✗
real	●	●	●	●	●	●	●	✗	✗	✗	✗	●	●	●		●	●	●	●	●	●	●	●	✗	✗	✗	✗	✗	●	✗	✗	✗
bigint	●	●	●	●	●	●	●	✗	✗	✗	✗	●	●	●	●		●	●	●	●	●	●	●	✗	✗	✗	✗	●	✗	✗	✗	✗
int(INT4)	●	●	●	●	●	●	●	✗	✗	✗	✗	●	●	●	●	●		●	●	●	●	●	●	✗	✗	✗	✗	●	✗	✗	✗	✗
smallint(INT2)	●	●	●	●	●	●	●	✗	✗	✗	✗	●	●	●	●	●	●		●	●	●	●	●	✗	✗	✗	✗	●	✗	✗	✗	✗
tinyint(INT1)	●	●	●	●	●	●	●	✗	✗	✗	✗	●	●	●	●	●	●	●		●	●	●	●	✗	✗	✗	✗	●	✗	✗	✗	✗
money	●	●	●	●	●	●	●	✗	✗	✗	✗	●	●	●	●	●	●	●	●		●	●	●	✗	✗	✗	✗	●	✗	✗	✗	✗
smallmoney	●	●	●	●	●	●	●	✗	✗	✗	✗	●	●	●	●	●	●	●	●	●		●	●	✗	✗	✗	✗	●	✗	✗	✗	✗
bit	●	●	●	●	●	●	●	✗	✗	✗	✗	●	●	●	●	●	●	●	●	●	●		●	✗	✗	✗	✗	●	✗	✗	✗	✗
timestamp	●	●	●	●	✗	✗	●	✗	✗	✗	✗	●	●	✗	✗	●	●	●	●	●	●	●		✗	●	✗	✗	✗	✗	✗	✗	✗
uniqueidentifier	●	●	●	●	●	●	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗		✗	✗	✗	✗	●	✗	✗	✗	✗
image	●	●	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	●	✗		✗	✗	✗	✗	✗	✗	✗
ntext	✗	✗	●	●	●	●	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗		●	✗	●	✗	✗	✗	✗
text	✗	✗	●	●	●	●	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗		●	✗	●	✗	✗	✗
sql_variant	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	✗	■	✗	✗	✗		✗	✗	✗
xml	■	■	■	■	■	■	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	○	○	○	○
CLR UDT	■	■	■	■	■	■	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	○	○	○	○
hierarchyid	■	■	■	■	■	■	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗

■ Explicit conversion

● Implicit conversion

✗ Conversion not allowed

◆ Requires explicit CAST to prevent the loss of precision or scale that might occur in an implicit conversion.

○ Implicit conversions between xml data types are supported only if the source or target is untyped xml. Otherwise, the conversion must be explicit.

En el gráfico anterior se muestran todas las conversiones explícitas e implícitas permitidas en SQL Server, pero el tipo de datos resultante de la conversión depende de la operación que se lleva a cabo:

En el caso de las conversiones explícitas, la instrucción misma determina el tipo de datos resultante.

En las conversiones implícitas, las instrucciones de asignación, como establecer el valor de una variable o insertar un valor en una columna, generarán el tipo de datos definido por la declaración de la variable o la definición de la columna.

En el caso de los operadores de comparación u otras expresiones, el tipo de datos resultante dependerá de las reglas de prioridad de los tipos de datos.

Sugerencia

Más adelante en esta sección puede consultar un ejemplo práctico sobre los efectos de la prioridad de los tipos de datos en las conversiones.

Al convertir entre `datetimeoffset` y los tipos de caracteres `char`, `nchar`, `nvarchar` y `varchar`, la parte del ajuste de zona horaria convertida siempre debe tener dígitos dobles para HH y MM. Por ejemplo, `-08:00`.

Puesto que los datos Unicode siempre usan un número par de bytes, preste atención al convertir datos `binary` o `varbinary` en o desde tipos de datos compatibles con Unicode. Por ejemplo, la siguiente conversión no devuelve el valor hexadecimal 41. Devuelve un valor hexadecimal de 4100:

```
SELECT CAST(CAST(0x41 AS nvarchar) AS varbinary);
```

Para más información, consulte [Compatibilidad con la intercalación y Unicode](#).

Tipos de datos de valor grande

Los tipos de datos de valor grande tienen el mismo comportamiento de conversión implícita y explícita que sus equivalentes más pequeños, en concreto los tipos de datos `nvarchar`, `varbinary` y `varchar`. Aun así, tenga en cuenta las directrices siguientes:

La conversión de datos `image` a datos `varbinary(max)` y viceversa funciona como una conversión implícita, al igual que las conversiones entre `text` y `varchar(max)` y entre `ntext` y `nvarchar(max)`.

La conversión de tipos de datos de valor grande, como `varchar(max)`, a un tipo de datos equivalente más pequeño, como `varchar`, es una conversión implícita, aunque se produce un truncamiento si el tamaño del valor grande supera la longitud especificada del tipo de datos más pequeño.

La conversión de `nvarchar`, `varbinary` o `varchar` a sus tipos de datos correspondientes de valor grande tiene lugar de forma implícita.

La conversión del tipo de datos `sql_variant` en los tipos de datos de valor grande es una conversión explícita.

Los tipos de datos de valor grande no se pueden convertir en el tipo de datos `sql_variant`.

Para más información sobre la conversión del tipo de datos `xml`, vea [Crear instancias de datos XML](#).

tipo de datos xml

Cuando se convierte de forma explícita o implícita el tipo de datos `xml` en un tipo de datos de cadena o binario, el contenido del tipo de datos `xml` se serializa en función de un conjunto de reglas definido. Para más información sobre estas reglas, vea [Definir la serialización de datos XML](#). Para más información sobre la conversión de otros tipos de datos al tipo de datos `xml`, vea [Crear instancias de datos XML](#).

Tipos de datos text e image

Los tipos de datos text e image no admiten la conversión automática de tipos de datos. Puede convertir explícitamente datos text en datos de caracteres y datos image en binary o varbinary, pero la longitud máxima es de 8000 bytes. Si intenta una conversión incorrecta (por ejemplo, intentando convertir una expresión de caracteres que incluye letras) a un tipo int, SQL Server devolverá un mensaje de error.

Intercalación de salida

Si las funciones CAST o CONVERT generan una cadena de caracteres y reciben una entrada de una cadena de caracteres, la salida tiene la misma intercalación y la misma etiqueta de intercalación que la entrada. Si la entrada no es una cadena de caracteres, la salida tiene la intercalación predeterminada de la base de datos y una etiqueta de intercalación coaccionable-predeterminada. Para obtener más información, vea Prioridad de intercalación (Transact-SQL).

Para asignar otra intercalación a la salida, aplique la cláusula COLLATE a la expresión de resultado de las funciones CAST o CONVERT. Por ejemplo:

```
SELECT CAST('abc' AS varchar(5)) COLLATE French_CS_AS;
```

Truncar y redondear resultados

Al convertir expresiones binarias o de caracteres (binary, char, nchar, nvarchar, varbinary o varchar) en una expresión de otro tipo de datos, la operación de conversión podría truncar los datos de salida, mostrar solo parcialmente los datos de salida o devolver un error. Estos casos se producen si el resultado es demasiado corto para mostrarse. Las conversiones a binary, char, nchar, nvarchar, varbinary o varchar se truncan, excepto aquellas que se muestran en la siguiente tabla.

De tipo de datos	En tipo de datos	Resultado
int, smallint o tinyint	char varchar	No se puede mostrar porque es demasiado corto
	nchar nvarchar r	Error ¹
money, smallmoney, numeric, decimal, float o real	char varchar	Error ¹
	nchar nvarchar r	Error ¹

¹ Error devuelto porque el resultado es demasiado corto para mostrarse.

SQL Server garantiza que solo las conversiones circulares (dicho de otra manera, las conversiones que convierten un tipo de datos en otro y después vuelven a convertirlo en el tipo de datos original) devuelvan los mismos valores en versiones diferentes. En el siguiente ejemplo se muestra una conversión circular:

```
DECLARE @myval DECIMAL(5, 2);
```

```
SET @myval = 193.57;
```

```
SELECT CAST(CAST(@myval AS VARBINARY(20)) AS DECIMAL(10, 5));
```

```
-- Or, using CONVERT
```

```
SELECT CONVERT(DECIMAL(10, 5), CONVERT(VARBINARY(20), @myval));
```

```
GO
```

Advertencia

No cree valores binary. Después, conviértalos a un tipo de datos de la categoría de datos numéricos. SQL Server no garantiza que el resultado de una conversión de tipos de datos decimal o numeric a binary sea idéntica en las distintas versiones de SQL Server.

En el siguiente ejemplo se muestra una expresión resultante demasiado corta para ser mostrada.

```
USE AdventureWorks2019;
```

```
GO
```

```
SELECT p.FirstName,
```

```
       p.LastName,
```

```
       SUBSTRING(p.Title, 1, 25) AS Title,
```

```
       CAST(e.SickLeaveHours AS CHAR(1)) AS [Sick Leave]
```

```
FROM HumanResources.Employee e
```

```
INNER JOIN Person.Person p
```

```
    ON e.BusinessEntityID = p.BusinessEntityID
```

```
WHERE NOT e.BusinessEntityID > 5;
```

```
GO
```

El conjunto de resultados es el siguiente:

FirstName LastName Title Sick Leave

Ken Sanchez NULL *

Terri Duffy NULL *

Roberto Tamburello NULL *

Rob Walters NULL *

Gail Erickson Ms. *

(5 row(s) affected)

Al convertir tipos de datos que difieren en los decimales, SQL Server devolverá a veces un valor de resultado truncado y, otras veces, un valor redondeado. En esta tabla se muestra el comportamiento.

De	A	Comportamiento
numeric	numeric	Round
numeric	int	Truncate
numeric	money	Round
money	int	Round
money	numeric	Round
float	int	Truncate
float	numeric	Ronda ¹
float	datetime	Round
datetime	int	Round

1 La conversión de los valores float que usan la notación científica a decimal o numeric se restringe únicamente a los valores con una precisión de 17 dígitos. Cualquier valor con una precisión mayor de 17 se redondea a cero.

Por ejemplo, los valores 10,6496 y -10,6496 se pueden truncar o redondear durante la conversión a los tipos int o numeric:

```
SELECT CAST(10.6496 AS INT) AS trunc1,  
       CAST(-10.6496 AS INT) AS trunc2,  
       CAST(10.6496 AS NUMERIC) AS round1,  
       CAST(-10.6496 AS NUMERIC) AS round2;
```

Los resultados de la consulta se muestran en la siguiente tabla:

trunc1	trunc2	round1	round2
10	-10	11	-11

Al convertir tipos de datos donde el tipo de datos de destino tiene menos decimales que el tipo de datos de origen, el valor se redondea. Por ejemplo, esta conversión devuelve \$10.3497:

```
SELECT CAST(10.3496847 AS money);
```

SQL Server devuelve un mensaje de error al convertir datos char, nchar, nvarchar o varchar no numéricos en datos decimal, float, int o numeric. SQL Server también devuelve un error cuando se convierte una cadena vacía (" ") en los tipos de datos numeric o decimal.

Determinadas conversiones de fecha y hora son no deterministas

Los estilos para los que la conversión de cadena a fecha y hora es no determinista son los siguientes:

Todos los estilos por debajo de 100 1

106

107

109

113

130

1 Con la excepción de los estilos 20 y 21

Para obtener más información, vea [Conversión no determinista de las cadenas de fecha literales en valores DATE](#).

Caracteres adicionales (pares suplentes)

A partir de SQL Server 2012 (11.x), si se usan intercalaciones de caracteres complementarios (SC), las operaciones CAST de nchar o nvarchar a un tipo nchar o nvarchar de menor longitud no se truncarán dentro de un par suplente. sino que lo hará antes del carácter suplementario. Por ejemplo, el fragmento de código siguiente deja @x con solo 'ab'. No hay espacio suficiente para albergar el carácter suplementario.

```
DECLARE @x NVARCHAR(10) = 'ab' + NCHAR(0x10000);
```

```
SELECT CAST(@x AS NVARCHAR(3));
```

Al usar intercalaciones de SC, el comportamiento de CONVERT es análogo al de CAST. Para más información, vea la sección Caracteres complementarios del artículo Compatibilidad con la intercalación y Unicode.

Soporte de compatibilidad

En versiones anteriores de SQL Server, el estilo predeterminado de las operaciones CAST y CONVERT en tipos de datos time y datetime2 es 121, a menos que se use otro tipo en una expresión de columna calculada. Para las columnas calculadas, el estilo predeterminado es 0. Este comportamiento afecta a las columnas calculadas cuando se crean, cuando se utilizan en las consultas que implican parametrización automática o cuando se usan en definiciones de restricciones.

En el nivel de compatibilidad 110 y posteriores, las operaciones CAST y CONVERT en los tipos de datos time y datetime2 siempre tienen el estilo predeterminado 121. Si una consulta se basa en el comportamiento anterior, use un nivel de compatibilidad menor de 110, o especifique explícitamente el estilo 0 en la consulta correspondiente.

Valor del nivel de compatibilidad	Estilo predeterminado para CAST y CONVERT ¹	Estilo predeterminado para la columna calculada
<110	121	0
> = 110	121	121

1 Excepto para las columnas calculadas

Actualizar la base de datos al nivel de compatibilidad 110 y posteriores no cambiará los datos de usuario que se hayan almacenado en disco. Debe corregir manualmente

estos datos según convenga. Por ejemplo, si usara SELECT INTO para crear una tabla de un origen que contuviera una expresión de columna calculada como la descrita anteriormente, se almacenarían los datos (si se usa el estilo 0) en lugar de la propia definición de columna calculada. Debe actualizar manualmente estos datos para que coincidan con el estilo 121.

Ejemplos

A. Uso de CAST y CONVERT

En estos ejemplos se recupera el nombre de aquellos productos que tienen un 3 como primer dígito del precio y se convierte sus valores ListPrice en int.

Use CAST:

```
USE AdventureWorks2019;
```

```
GO
```

```
SELECT SUBSTRING(Name, 1, 30) AS ProductName,
```

```
    ListPrice
```

```
FROM Production.Product
```

```
WHERE CAST(ListPrice AS INT) LIKE '33%';
```

```
GO
```

Use CONVERT:

```
USE AdventureWorks2019;
```

```
GO
```

```
SELECT SUBSTRING(Name, 1, 30) AS ProductName,
```

```
    ListPrice
```

```
FROM Production.Product
```

```
WHERE CONVERT(INT, ListPrice) LIKE '33%';  
GO
```

El conjunto de resultados es el siguiente: El conjunto de resultados de ejemplo es el mismo para CAST y CONVERT.

ProductName	ListPrice

LL Road Frame - Black, 58	337.22
LL Road Frame - Black, 60	337.22
LL Road Frame - Black, 62	337.22
LL Road Frame - Red, 44	337.22
LL Road Frame - Red, 48	337.22
LL Road Frame - Red, 52	337.22
LL Road Frame - Red, 58	337.22
LL Road Frame - Red, 60	337.22
LL Road Frame - Red, 62	337.22
LL Road Frame - Black, 44	337.22
LL Road Frame - Black, 48	337.22
LL Road Frame - Black, 52	337.22
Mountain-100 Black, 38	3374.99
Mountain-100 Black, 42	3374.99
Mountain-100 Black, 44	3374.99
Mountain-100 Black, 48	3374.99
HL Road Front Wheel	330.06
LL Touring Frame - Yellow, 62	333.42
LL Touring Frame - Blue, 50	333.42

LL Touring Frame - Blue, 54	333.42
LL Touring Frame - Blue, 58	333.42
LL Touring Frame - Blue, 62	333.42
LL Touring Frame - Yellow, 44	333.42
LL Touring Frame - Yellow, 50	333.42
LL Touring Frame - Yellow, 54	333.42
LL Touring Frame - Yellow, 58	333.42
LL Touring Frame - Blue, 44	333.42
HL Road Tire	32.60

(28 rows affected)

B. Uso de CAST con operadores aritméticos

En este ejemplo se calcula una única columna (Computed) mediante la división de las ventas anuales hasta la fecha (SalesYTD) entre el porcentaje de la comisión (CommissionPCT). Este valor se redondea al número entero más cercano y luego se CAST en un tipo de datos int.

```
USE AdventureWorks2019;
```

```
GO
```

```
SELECT CAST(ROUND(SalesYTD / CommissionPCT, 0) AS INT) AS Computed
```

```
FROM Sales.SalesPerson
```

```
WHERE CommissionPCT != 0;
```

```
GO
```

El conjunto de resultados es el siguiente:

Computed

379753754

346698349

257144242

176493899

281101272

0

301872549

212623750

298948202

250784119

239246890

101664220

124511336

97688107

(14 row(s) affected)

C. Uso de CAST para concatenar

En este ejemplo se concatenan expresiones que no son de caracteres usando CAST.

Usa la base de datos AdventureWorksDW2019.

```
SELECT 'The list price is ' + CAST(ListPrice AS VARCHAR(12)) AS ListPrice  
FROM dbo.DimProduct  
WHERE ListPrice BETWEEN 350.00 AND 400.00;
```

El conjunto de resultados es el siguiente:

ListPrice

The list price is 357.06

The list price is 364.09

The list price is 364.09

The list price is 364.09

The list price is 364.09

D. Uso de CAST para obtener texto más legible

En este ejemplo se usa CAST en la lista SELECT para convertir la columna Name en una columna de tipo char(10). Usa la base de datos AdventureWorksDW2019.

```
SELECT DISTINCT CAST(EnglishProductName AS CHAR(10)) AS Name,  
    ListPrice  
FROM dbo.DimProduct  
WHERE EnglishProductName LIKE 'Long-Sleeve Logo Jersey, M';  
GO
```

El conjunto de resultados es el siguiente:

Name ListPrice

Long-Sleev 31.2437

Long-Sleev 32.4935

Long-Sleeve 49.99

E. Uso de CAST con la cláusula LIKE

En este ejemplo se convierten los valores SalesYTD de la columna money al tipo de datos int y, después, al tipo de datos char(20), de modo que la cláusula LIKE pueda usarlo.

```
USE AdventureWorks2019;
```

```
GO
```

```
SELECT p.FirstName,
```

```
       p.LastName,
```

```
       s.SalesYTD,
```

```
       s.BusinessEntityID
```

```
FROM Person.Person AS p
```

```
INNER JOIN Sales.SalesPerson AS s
```

```
    ON p.BusinessEntityID = s.BusinessEntityID
```

```
WHERE CAST(CAST(s.SalesYTD AS INT) AS CHAR(20)) LIKE '2%';
```

```
GO
```

El conjunto de resultados es el siguiente:

FirstName	LastName	SalesYTD	BusinessEntityID
Tsvi	Reiter	2811012.7151	279
Syed	Abbas	219088.8836	288
Rachel	Valdez	2241204.0424	289

(3 row(s) affected)

F. Uso de CONVERT o CAST con XML con tipo

En estos ejemplos se muestra el uso de CONVERT para convertir datos a XML con tipo mediante Tipo de datos XML y columnas (SQL Server).

En este ejemplo se convierte una cadena con espacios en blanco, texto y marcado en XML con tipo y se quitan todos los espacios en blanco insignificantes (espacios en blanco de límite entre los nodos):

```
SELECT CONVERT(XML, '<root><child/></root>')
```

En este ejemplo se convierte una cadena similar con espacios en blanco, texto y marcado en XML con tipo y se conservan los espacios en blanco insignificantes (espacios en blanco de límite entre los nodos):

```
SELECT CONVERT(XML, '<root>      <child/>      </root>', 1)
```

En este ejemplo se convierte una cadena con espacios en blanco, texto y marcado en XML con tipo:

```
SELECT  
CAST('<Name><FName>Carol</FName><LName>Elliot</LName></Name>' AS  
XML)
```

Vea [Crear instancias de datos XML](#) para ver más ejemplos.

G. Uso de CAST y CONVERT con datos de fecha y hora

A partir de los valores GETDATE(), en este ejemplo se muestran la fecha y la hora actuales, se usa CAST para cambiarlas a un tipo de datos de caracteres y, después, se usa CONVERT para mostrar la fecha y la hora en el formato ISO 8601.

```
SELECT GETDATE() AS UnconvertedDateTime,  
       CAST(GETDATE() AS NVARCHAR(30)) AS UsingCast,  
       CONVERT(NVARCHAR(30), GETDATE(), 126) AS UsingConvertTo_ISO8601;  
GO
```

El conjunto de resultados es el siguiente:

UnconvertedDateTime	UsingCast	UsingConvertTo_ISO8601

2022-04-18 09:58:04.570	Apr 18 2022 9:58AM	2022-04-18T09:58:04.570

(1 row(s) affected)

Este ejemplo es lo opuesto, aproximadamente, al ejemplo anterior. En este ejemplo, se muestra la fecha y la hora como datos de caracteres, se usa CAST para cambiar los datos de caracteres al tipo de datos datetime y, luego, se usa CONVERT para cambiar los datos de caracteres al tipo de datos datetime.

```
SELECT '2006-04-25T15:50:59.997' AS UnconvertedText,  
       CAST('2006-04-25T15:50:59.997' AS DATETIME) AS UsingCast,  
       CONVERT(DATETIME, '2006-04-25T15:50:59.997', 126) AS  
UsingConvertFrom_ISO8601;  
GO
```

El conjunto de resultados es el siguiente:

UnconvertedText	UsingCast	UsingConvertFrom_ISO8601

2006-04-25T15:50:59.997	2006-04-25 15:50:59.997	2006-04-25 15:50:59.997

(1 row(s) affected)

H. Uso de CONVERT con datos binarios y de caracteres

En estos ejemplos se muestran los resultados de la conversión de datos binarios y de caracteres usando estilos diferentes.

--Convert the binary value 0x4E616d65 to a character value.

```
SELECT CONVERT(CHAR(8), 0x4E616d65, 0) AS [Style 0, binary to character];
```

El conjunto de resultados es el siguiente:

Style 0, binary to character

Name

(1 row(s) affected)

En este ejemplo se muestra que Style 1 puede forzar el truncamiento del resultado. Los caracteres 0x del conjunto de resultados fuerzan el truncamiento.

```
SELECT CONVERT(CHAR(8), 0x4E616d65, 1) AS [Style 1, binary to character];
```

El conjunto de resultados es el siguiente:

Style 1, binary to character

0x4E616D

(1 row(s) affected)

En este ejemplo se muestra que Style 2 no trunca el resultado, ya que este no incluye los caracteres 0x.

```
SELECT CONVERT(CHAR(8), 0x4E616d65, 2) AS [Style 2, binary to character];
```

El conjunto de resultados es el siguiente:

Style 2, binary to character

4E616D65

(1 row(s) affected)

Convierta el valor del carácter 'Name' en un valor binario.

```
SELECT CONVERT(BINARY(8), 'Name', 0) AS [Style 0, character to binary];
```

El conjunto de resultados es el siguiente:

Style 0, character to binary

0x4E616D6500000000

(1 row(s) affected)

```
SELECT CONVERT(BINARY(4), '0x4E616D65', 1) AS [Style 1, character to binary];
```


El conjunto de resultados es el siguiente:

Style 1, character to binary

0x4E616D65

(1 row(s) affected)

```
SELECT CONVERT(BINARY(4), '4E616D65', 2) AS [Style 2, character to binary];
```

El conjunto de resultados es el siguiente:

Style 2, character to binary

0x4E616D65

(1 row(s) affected)

I. Convierte tipos de datos de fecha y hora

En este ejemplo se muestra la conversión de los tipos de datos date, time y datetime.

```
DECLARE @d1 DATE,
```

```
        @t1 TIME,
```

```
        @dt1 DATETIME;
```

```
SET @d1 = GETDATE();
```

```
SET @t1 = GETDATE();
```

```
SET @dt1 = GETDATE();
```

```
SET @d1 = GETDATE();
```

-- When converting date to datetime the minutes portion becomes zero.

```
SELECT @d1 AS [DATE],  
       CAST(@d1 AS DATETIME) AS [date as datetime];
```

-- When converting time to datetime the date portion becomes zero

-- which converts to January 1, 1900.

```
SELECT @t1 AS [TIME],  
       CAST(@t1 AS DATETIME) AS [time as datetime];
```

-- When converting datetime to date or time non-applicable portion is dropped.

```
SELECT @dt1 AS [DATETIME],  
       CAST(@dt1 AS DATE) AS [datetime as date],  
       CAST(@dt1 AS TIME) AS [datetime as time];
```

Asegúrese de que los valores están dentro de un intervalo compatible al considerar una conversión de date a datetime o datetime2. El valor de año mínimo para datetime es 1753, mientras que el valor de año mínimo es 0001 para date y datetime2.

```
DECLARE @d1 DATE, @dt1 DATETIME , @dt2 DATETIME2
```

```
SET @d1 = '1492-08-03'
```

--This is okay; Minimum YYYY for DATE is 0001

```
SET @dt2 = CAST(@d1 AS DATETIME2)
```

--This is okay; Minimum YYYY for DATETIME2 IS 0001

```
SET @dt1 = CAST(@d1 AS DATETIME)
```

--This will error with (Msg 242) "The conversion of a date data type to a datetime data type resulted in an out-of-range value."

--Minimum YYYY for DATETIME is 1753

J. Uso de CONVERT con datos de datetime en formatos diferentes

A partir de los valores GETDATE(), en este ejemplo se usa CONVERT para mostrar todos los estilos de fecha y hora en la sección Estilos de fecha y hora de este artículo.

Formato de número	Ejemplo de consulta	Resultado de muestra
0	SELECT CONVERT(NVARCHAR, GETDATE(), 0)	23 ago de 2019 13:39
1	SELECT CONVERT(NVARCHAR, GETDATE(), 1)	08/23/19
2	SELECT CONVERT(NVARCHAR, GETDATE(), 2)	19.08.23
3	SELECT CONVERT(NVARCHAR, GETDATE(), 3)	23/08/19
4	SELECT CONVERT(NVARCHAR, GETDATE(), 4)	23.08.19
5	SELECT CONVERT(NVARCHAR, GETDATE(), 5)	23-08-19

6	SELECT CONVERT(NVARCHAR, GETDATE(), 6)	23 ago 19
7	SELECT CONVERT(NVARCHAR, GETDATE(), 7)	Ago 23, 19
8, 24 o 108	SELECT CONVERT(NVARCHAR, GETDATE(), 8)	13:39:17
9 o 109	SELECT CONVERT(NVARCHAR, GETDATE(), 9)	23 ago 2019 13:39:17:090
10	SELECT CONVERT(NVARCHAR, GETDATE(), 10)	08-23-19
11	SELECT CONVERT(NVARCHAR, GETDATE(), 11)	19/08/23
12	SELECT CONVERT(NVARCHAR, GETDATE(), 12)	190823
13 o 113	SELECT CONVERT(NVARCHAR, GETDATE(), 13)	23 ago 2019 13:39:17:090
14 o 114	SELECT CONVERT(NVARCHAR, GETDATE(), 14)	13:39:17:090

20 o 120	SELECT CONVERT(NVARCHAR, GETDATE(), 20)	2019-08-23 13:39:17
21 o 25 o 121	SELECT CONVERT(NVARCHAR, GETDATE(), 21)	2019-08-23 13:39:17.090
22	SELECT CONVERT(NVARCHAR, GETDATE(), 22)	08/23/19 13:39:17
23	SELECT CONVERT(NVARCHAR, GETDATE(), 23)	2019-08-23
101	SELECT CONVERT(NVARCHAR, GETDATE(), 101)	08/23/2019
102	SELECT CONVERT(NVARCHAR, GETDATE(), 102)	2019.08.23
103	SELECT CONVERT(NVARCHAR, GETDATE(), 103)	23/08/2019
104	SELECT CONVERT(NVARCHAR, GETDATE(), 104)	23.08.2019
105	SELECT CONVERT(NVARCHAR, GETDATE(), 105)	23-08-2019

106	SELECT CONVERT(NVARCHAR, GETDATE(), 106)	23 ago 2019
107	SELECT CONVERT(NVARCHAR, GETDATE(), 107)	23 de agosto de 2019
110	SELECT CONVERT(NVARCHAR, GETDATE(), 110)	08-23-2019
111	SELECT CONVERT(NVARCHAR, GETDATE(), 111)	2019/08/23
112	SELECT CONVERT(NVARCHAR, GETDATE(), 112)	20190823
113	SELECT CONVERT(NVARCHAR, GETDATE(), 113)	23 ago 2019 13:39:17.090
120	SELECT CONVERT(NVARCHAR, GETDATE(), 120)	2019-08-23 13:39:17
121	SELECT CONVERT(NVARCHAR, GETDATE(), 121)	2019-08-23 13:39:17.090
126	SELECT CONVERT(NVARCHAR, GETDATE(), 126)	2019-08-23T13:39:17.09

127	SELECT CONVERT(NVARCHAR, GETDATE(), 127)	2019-08-23T13:39:17.09
130	SELECT CONVERT(NVARCHAR, GETDATE(), 130)	22 13:39:17.090 1440 Pذو الحجة
131	SELECT CONVERT(NVARCHAR, GETDATE(), 131)	22/12/1440 13:39:17.090

K. Efectos de la prioridad de los tipos de datos en las conversiones permitidas

En el siguiente ejemplo se define una variable de tipo varchar(10), se asigna un valor de tipo entero a la variable y, luego, se selecciona una concatenación de la variable con una cadena.

```
DECLARE @string VARCHAR(10);
SET @string = 1;
SELECT @string + ' is a string.' AS Result
```

El conjunto de resultados es el siguiente:

Result

1 is a string.

El valor int de 1 se ha convertido a varchar.

En este ejemplo se muestra una consulta similar con una variable int en su lugar:

```
DECLARE @notastring INT;
```

```
SET @notastring = '1';  
SELECT @notastring + ' is not a string.' AS Result
```

En este caso, la instrucción SELECT producirá el siguiente error:

Msg 245, Level 16, State 1, Line 3

Conversion failed when converting the varchar value ' is not a string.' to data type int.

Para evaluar la expresión @notastring + ' is not a string.', SQL Server necesita seguir las reglas de prioridad del tipo de datos para completar la conversión implícita antes de que se pueda calcular el resultado de la expresión. Dado que int tiene una prioridad más alta que varchar, SQL Server intenta convertir la cadena a un entero y produce un error porque esta cadena no se puede convertir a un entero.

Si proporcionamos una cadena que se pueda convertir, la instrucción se ejecutará correctamente, como se ve en el ejemplo siguiente:

```
DECLARE @notastring INT;  
SET @notastring = '1';  
SELECT @notastring + '1'
```

En este caso, la cadena '1' se puede convertir al valor entero 1, por lo que esta instrucción SELECT devuelve el valor 2. Cuando los tipos de datos proporcionados son enteros, el operador + se convierte en un operador matemático de suma, en lugar de una concatenación de cadena.

L. Uso de CAST y CONVERT

En este ejemplo se recupera el nombre de aquellos productos que tienen un 3 como primer dígito del precio y se convierte su ListPrice en int. Usa la base de datos AdventureWorksDW2019.

```
SELECT EnglishProductName AS ProductName, ListPrice
FROM dbo.DimProduct
WHERE CAST(ListPrice AS int) LIKE '3%';
```

En este ejemplo se muestra la misma consulta, pero usando CONVERT en vez de CAST. Usa la base de datos AdventureWorksDW2019.

```
SELECT EnglishProductName AS ProductName, ListPrice
FROM dbo.DimProduct
WHERE CONVERT(INT, ListPrice) LIKE '3%';
```

M. Uso de CAST con operadores aritméticos

En este ejemplo se calcula un único valor de columna dividiendo el precio por unidad de producto (UnitPrice) entre el porcentaje de descuento (UnitPriceDiscountPct). Este resultado se redondea al número entero más cercano y, por último, se convierte al tipo de datos int. En este ejemplo se usa la base de datos AdventureWorksDW2019.

```
SELECT ProductKey, UnitPrice, UnitPriceDiscountPct,
       CAST(ROUND (UnitPrice*UnitPriceDiscountPct,0) AS int) AS DiscountPrice
FROM dbo.FactResellerSales
WHERE SalesOrderNumber = 'SO47355'
       AND UnitPriceDiscountPct > .02;
```

El conjunto de resultados es el siguiente:

ProductKey	UnitPrice	UnitPriceDiscountPct	DiscountPrice
------------	-----------	----------------------	---------------

-----	-----	-----	-----
-------	-------	-------	-------

323	430.6445	0.05	22
213	18.5043	0.05	1
456	37.4950	0.10	4
456	37.4950	0.10	4
216	18.5043	0.05	1

Hora Uso de CAST con la cláusula LIKE

En este ejemplo se convierte ListPrice de la columna money a un tipo int y, luego, a un tipo char(20) , de modo que la cláusula LIKE pueda usarla. En este ejemplo se usa la base de datos AdventureWorksDW2019.

```
SELECT EnglishProductName AS Name, ListPrice
FROM dbo.DimProduct
WHERE CAST(CAST(ListPrice AS INT) AS CHAR(20)) LIKE '2%';
```

O. Uso de CAST y CONVERT con datos de fecha y hora

En este ejemplo se muestra la fecha y la hora actuales, se usa CAST para cambiarlas a un tipo de datos de caracteres y, por último, se usa CONVERT para mostrar la fecha y la hora en el formato ISO 8601. En este ejemplo se usa la base de datos AdventureWorksDW2019.

```
SELECT TOP(1)
    SYSDATETIME() AS UnconvertedDateTime,
    CAST(SYSDATETIME() AS NVARCHAR(30)) AS UsingCast,
    CONVERT(NVARCHAR(30), SYSDATETIME(), 126) AS UsingConvertTo_ISO8601
```

```
FROM dbo.DimCustomer;
```

El conjunto de resultados es el siguiente:

UnconvertedDateTime	UsingCast	UsingConvertTo_ISO8601
07/20/2010 1:44:31 PM	2010-07-20 13:44:31.5879025	2010-07-20T13:44:31.5879025

Este ejemplo es lo opuesto, aproximadamente, al ejemplo anterior. En este ejemplo, se muestra la fecha y la hora como datos de caracteres, se usa CAST para cambiar los datos de caracteres al tipo de datos datetime y, luego, se usa CONVERT para cambiar los datos de caracteres al tipo de datos datetime. En este ejemplo se usa la base de datos AdventureWorksDW2019.

```
SELECT TOP(1)
    '2010-07-25T13:50:38.544' AS UnconvertedText,
    CAST('2010-07-25T13:50:38.544' AS DATETIME) AS UsingCast,
    CONVERT(DATETIME, '2010-07-25T13:50:38.544', 126) AS
    UsingConvertFrom_ISO8601
FROM dbo.DimCustomer;
```

El conjunto de resultados es el siguiente:

UnconvertedText	UsingCast	UsingConvertFrom_ISO8601
2010-07-25T13:50:38.544	07/25/2010 1:50:38 PM	07/25/2010 1:50:38 PM

ISNULL

Sustituye el valor NULL por el valor especificado.

ISNULL (check_expression , replacement_value)

Argumentos

check_expression

Es la expresión que se va a comprobar si es NULL. check_expression puede ser de cualquier tipo.

replacement_value

Es la expresión que se devuelve si check_expression es NULL. valor_de_reemplazo debe ser de un tipo que se pueda convertir implícitamente en el tipo de expresión_de_comprobación.

Tipos de valor devuelto

Devuelve el mismo tipo que check_expression. Si se proporciona un literal NULL como check_expression, devuelve el tipo de datos de replacement_value. Si se proporciona un literal NULL como check_expression y no se proporciona replacement_value, se devuelve un int.

Observaciones

El valor de check_expression se devuelve si no es NULL; de lo contrario, se devuelve replacement_value después de convertirse de forma implícita al tipo de check_expression, si los tipos son diferentes. replacement_value se puede truncar si replacement_value es mayor que check_expression.

Nota

Use COALESCE (Transact-SQL) para devolver el primer valor distinto de NULL.

Ejemplos

A. Usar ISNULL con AVG

En el ejemplo siguiente se busca el promedio del peso de todos los productos. Sustituye el valor 50 para todas las entradas NULL en la columna Weight de la tabla Product.

```
USE AdventureWorks2012;  
GO  
SELECT AVG(ISNULL(Weight, 50))  
FROM Production.Product;  
GO
```

El conjunto de resultados es el siguiente:

59.79

(1 row(s) affected)

B. Usar ISNULL

En el siguiente ejemplo se selecciona la descripción, el porcentaje de descuento, la cantidad mínima y la cantidad máxima de todas las ofertas especiales de AdventureWorks2012. Si la cantidad máxima de una oferta especial determinada es NULL, el valor de MaxQty mostrado en el conjunto de resultados es 0.00.

```
USE AdventureWorks2012;  
GO  
SELECT Description, DiscountPct, MinQty, ISNULL(MaxQty, 0.00) AS 'Max Quantity'  
FROM Sales.SpecialOffer;
```

El conjunto de resultados es el siguiente

Descripción	DiscountPct	MinQty	Cantidad máxima
No Discount	0.00	0	0
Volume Discount	0.02	11	14
Volume Discount	0,05	15	4
Volume Discount	0,10	25	0
Volume Discount	0,15	41	0
Volume Discount	0,20	61	0
Mountain-100 Cl	0.35	0	0
Sport Helmet Di	0,10	0	0
Road-650 Overst	0,30	0	0
Mountain Tire S	0.50	0	0
Sport Helmet Di	0,15	0	0
LL Road Frame S	0.35	0	0
Touring-3000 Pr	0,15	0	0
Touring-1000 Pr	0,20	0	0
Half-Price Peda	0.50	0	0
Mountain-500 Si	0.40	0	0

(16 row(s) affected)

C. Comprobar si hay valores NULL en una cláusula WHERE

No utilice ISNULL para buscar los valores NULL. Use IS NULL en su lugar. En el ejemplo siguiente se buscan todos los productos que tienen NULL en la columna de peso. Tenga en cuenta el espacio entre IS y NULL

```
USE AdventureWorks2012;  
GO  
SELECT Name, Weight  
FROM Production.Product  
WHERE Weight IS NULL;  
GO
```

D. Usar ISNULL con AVG

En el ejemplo siguiente se busca el promedio del peso de todos los productos en una tabla de ejemplo. Sustituye el valor 50 para todas las entradas NULL en la columna Weight de la tabla Product.

```
-- Uses AdventureWorks
```

```
SELECT AVG(ISNULL(Weight, 50))  
FROM dbo.DimProduct;
```

El conjunto de resultados es el siguiente:

52.88

E. Usar ISNULL

En el ejemplo siguiente se usa ISNULL para comprobar los valores NULL en la columna MinPaymentAmount y mostrar el valor 0.00 para esas filas.

```
-- Uses AdventureWorks
```

```
SELECT ResellerName,
```

```
ISNULL(MinPaymentAmount,0) AS MinimumPayment
FROM dbo.DimReseller
ORDER BY ResellerName;
```

A continuación se muestra un conjunto parcial de resultados.

ResellerName	MinimumPayment
Una asociación de bicicleta	0.0000
Una tienda de bicicletas	0.0000
Una tienda de bicis	0.0000
Una gran empresa de bicicletas	0.0000
La típica tienda de bicicletas	200.0000
Servicio y ventas aceptables	0.0000

F. Usar IS NULL para probar NULL en una cláusula WHERE

En el ejemplo siguiente se buscan todos los productos que tienen NULL en la columna Weight. Tenga en cuenta el espacio entre IS y NULL.

-- Uses AdventureWorks

```
SELECT EnglishProductName, Weight
FROM dbo.DimProduct
WHERE Weight IS NULL;
```


Bibliografía y Webgrafía

Jacobs, P. (2019). SQL: Guía completa para principiantes de la programación SQL con ejercicios y estudios de casos (Libro En Español/SQL Spanish Book Version) (Spanish Edition). Independently Published.

Moestl Vasilik, S. (2016). Problemas prácticos de SQL: 57 desafíos iniciales, intermedios y avanzados para que los resuelvas utilizando un enfoque de «aprender haciendo». Packt Publishing.

Microsoft. (2023, 20 de enero). Introducción a T-SQL. [Curso en línea]. Microsoft Learn. Recuperado de <https://learn.microsoft.com/en-us/training/modules/introduction-to-transact-sql/>

Microsoft. (2023, 20 de enero). Introducción a las consultas con T-SQL. [Ruta de formación]. Microsoft Learn. Recuperado de <https://learn.microsoft.com/en-us/training/paths/get-started-querying-with-transact-sql/>

Microsoft. (2023, 20 de enero). Introducción a la programación T-SQL. [Curso en línea]. Microsoft Learn. Recuperado de <https://learn.microsoft.com/en-us/training/modules/get-started-transact-sql-programming/>
<https://learn.microsoft.com/es-es/sql/t-sql/language-reference?view=sql-server-ver16>

Glosario

Cláusulas SQL: Son componentes fundamentales de una consulta SQL que permiten filtrar, ordenar y limitar los resultados obtenidos de una base de datos.

Comandos SQL para manipulación de registros: Son instrucciones utilizadas en el lenguaje SQL para realizar operaciones como inserción, actualización y eliminación de registros en una base de datos.

Funciones de Agregado: Son funciones en SQL que permiten realizar cálculos sobre grupos de datos, como sumas, promedios, máximos y mínimos.

Funciones de Fecha: Son funciones en SQL que facilitan la manipulación y cálculo de fechas, permitiendo realizar operaciones como sumar días, extraer partes de una fecha, entre otros.

Funciones de Texto: Son funciones en SQL que permiten manipular cadenas de texto, como concatenar, convertir a mayúsculas o minúsculas, entre otras operaciones.

Operadores Lógicos: Son símbolos o palabras clave utilizadas en SQL para combinar condiciones en una consulta, como AND, OR y NOT, para obtener resultados basados en condiciones lógicas.

Orden de Procesamiento de Consultas: Es el proceso en el cual se ejecutan y evalúan las diferentes cláusulas y operaciones en una consulta SQL para obtener el resultado final.

Tipos de Datos: Son categorías que definen el tipo de valor que puede almacenarse en una columna de una tabla en una base de datos, como enteros, cadenas de texto, fechas, valores booleanos, entre otros.